本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.6.2 (https://github.com/funstory-ai/BabelDOC) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

iSAM2: 基于贝叶斯树的增量平滑与映射

迈克尔·凯斯、霍多尔·约翰逊、理查德·罗伯茨、维奥雷拉·伊拉、约翰·伦纳德和弗兰克·德拉尔

摘要

我们提出了一种新的数据结构——贝叶斯树，它为理解现有图模型推理算法及其与稀疏矩阵分解方法之间的联系提供了算法基础。类似于团树，贝叶斯树编码了一个分解的概率密度，但与团树不同，它是定向的，并且更自然地映射到同步定位与映射（SLAM）问题的平方根信息矩阵。在本文中，我们强调了我们的新数据结构提供的三个见解。首先，贝叶斯树更好地理解了概率密度下的矩阵分解。其次，我们展示了矩阵分解的抽象更新如何转化为贝叶斯树及其条件密度的简单编辑。第三，我们将贝叶斯树应用于获得一种全新的稀疏非线性增量优化算法，命名为iSAM2，该算法通过增量变量重排序和流体重线性化实现了效率提升，无需周期性的批量步骤。我们详细分析了iSAM2的各种特性，并在一系列真实和模拟数据集上展示了我们的算法在质量和效率方面均优于其他最近的映射算法。

关键词： 图模型、团树、连接树、概率推理、稀疏线性代数、非线性优化、平滑与映射、SLAM

1 引言

概率推理算法在机器人学中非常重要，应用范围广泛，从用于构建世界几何模型的同步定位与地图构建（SLAM），到用于人机交互的人群跟踪。我们的研究主要在大规模SLAM领域，因此全文将以此为例进行说明。SLAM是自主机器人的核心能力之一，它不仅为规划与操作等许多其他重要任务提供必要数据，还直接应用于导航、探索和3D建模等场景。传感器测量中的不确定性使得概率推理算法成为SLAM的首选。在线操作对于大多数实际应用至关重要，因此我们的工作专注于高效的增量在线算法。

从图模型视角看待SLAM中的概率推理有着悠久的历史（Brooks，1985），尤其在过去几年中，它特别推动了多项新颖且令人兴奋的发展（Paskin，2003；Folkesson和Christensen，2004；Frese等人，2005；Frese，2006；Folkesson和Christensen，2007；Ranganathan等人，2007）。Paskin（2003）提出了薄连接树滤波器（TJTF），它直接基于图模型提供了一种增量式解决方案。然而，滤波器应用已知在应用于本质上非线性的SLAM问题时是不一致的（Julier和Uhlmann，2001），即大量实验的平均值会偏离真实解。相比之下，完整*SLAM* (Thrun等人，2005)保留了所有机器人位姿，并能提供精确解，不会出现不一致问题。Folkesson和Christensen（2004）提出了基于图的完整SLAM解决方案——图SLAM，该方案通过局部减少变量数量来降低复杂度。更相关的是，Frese（2006）的Treemap在树的节点内执行QR分解。Loopy SAM（Ranganathan等人，2007）直接将回环信念传播应用于SLAM图。

稀疏线性代数视角在平滑与映射（SAM）中已被探讨（Dellaert, 2005; Dellaert和Kaess, 2006; Kaess等人，2007, 2008）。与平滑相关的矩阵通常非常稀疏，并且可以比分解密集矩阵的立方复杂度做得更好（Krauthausen等人，2006）。Kaess等人（2008）提出了增量平滑与映射（iSAM），它执行<样式 id='1'>快速增量更新</样式>平方根信息矩阵，但能够随时计算完整地图和轨迹。新测量值使用矩阵更新方程（Gill等人, 1974; Gentleman, 1973; Golub和Loan, 1996）添加，以便重用先前计算出的平方根信息矩阵的组件。然而，为了保持高效和一致性，iSAM需要周期性的批量步骤来允许变量重排序和重新线性化，这是昂贵的，并削弱了算法的预期在线特性。

草稿稿件，2011年4月6日。提交至IJRR。M. Kaess、H. Johannsson和J. Leonard就职于麻省理工学院（MIT）计算机科学与人工智能实验室（CSAIL），美国马萨诸塞州剑桥市02139 {kaess, hordurj, jleonard}@mit.edu。R. Roberts、V. Ila和F. Dellaert就职于佐治亚理工学院交互计算学院，美国佐治亚州亚特兰大市30332 {richard, vila, frank}@cc.gatech.edu。部分工作于2010年12月在新加坡举行的机器人算法基础国际研讨会进行展示，部分工作于2011年5月在中国上海举行的机器人与自动化国际会议进行展示。

线性化，这是昂贵的，并削弱了算法的预期在线特性。

为结合图模型和稀疏线性代数的优势，我们提出了一种新的数据结构，即贝叶斯树，该结构首次由Kaess等人于2010年提出。我们的方法基于将矩阵分解视为将因子图转换为贝叶斯网络，而贝叶斯网络是图模型中平方根信息矩阵的等价物。在贝叶斯网络中执行边缘化和优化通常并不容易。然而，通过消除/分解得到的贝叶斯网络结果是和弦的，众所周知，和弦贝叶斯网络可以转换为树状图模型，在这些模型中这些操作很容易。这种数据结构与团树(Pothen和Sun, 1992; 布莱尔和皮尔逊, 1993; Koller 和 Friedman, 2009) 相似，在人工智能文献中也被称为连接树(考威尔等人, 1999)，这些结构已被用于SLAM中的分布式推理(Dellaert等人, 2005; Paskin, 2003)。然而，我们在此提出的新数据结构——贝叶斯树，是有向的，并且更自然地对应于线性代数中QR分解的结果，使我们能够以树中的条件概率密度进行分析。我们进一步证明增量推理对应于对该树的一个简单编辑，并提出了一个新的增量变量排序策略。

利用这种新的数据结构和从中获得的洞察，我们提出了iSAM2，一种新颖的增量精确推理方法，允许增量重排序和即时重线性化。 iSAM2最初由Kaess等人（2011年）提出，通过利用图模型和稀疏线性代数视角之间联系的相关见解，扩展了我们最初的iSAM算法。据我们所知，这是一种完全新颖的方法，能够在增量设置中为稀疏非线性优化问题提供高效且精确的解决方案，其应用范围已超越SLAM。虽然标准的非线性优化方法会反复求解线性批量问题以更新线性化点，但我们的基于贝叶斯树的算法允许对一组减少的变量进行流体重线性化，这转化为更高的效率，同时保持稀疏性和完全的准确性。图1展示了小型SLAM序列的贝叶斯树示例。当机器人探索环境时，新的测量值仅影响树的一部分，并且仅重新计算这些部分。

本文提供了对 iSAM2 的详细评估，并与其他最先进的 SLAM 算法进行了比较。我们探讨了不同的变量排序策略对 iSAM2 性能的影响。此外，我们评估了重新线性化和更新阈值的效果，作为速度和精度之间的权衡，表明在仍然获得几乎精确解的同时，可以实现巨大的计算节省。最后，我们使用一系列 2D 和 3D、模拟和真实世界、仅基于位姿和基于地标的数据集，与其他 SLAM 算法在计算和精度方面进行了详细比较。

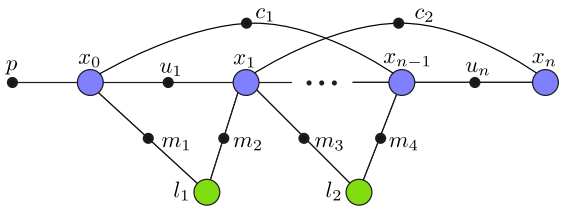


图2： SLAM问题的因子图（Kschischang等人，2001年）公式，其中变量节点显示为大圆圈，因子节点（测量值）显示为小实心圆圈。显示的因子包括里程计测量u、先验p、回环闭合约束c和地标测量m。特殊情况包括位姿图公式（不包含l和m）和基于地标的SLAM（不包含c）。请注意，因子图可以表示任何成本函数，该函数涉及一个、两个或多个变量（例如标定）。

2 问题陈述

这项工作重点是如何以增量式和实时的方式高效地解决非线性估计问题。增量式意味着每当添加新的测量值时，都必须获得更新的估计，以反映迄今为止所有测量值所能推导出的最精确的环境模型。在线式意味着估计必须在机器人运行期间获得，而不是在机器人任务完成后进行批量计算，因为机器人需要该估计来进行导航和规划以实现给定目标。

我们使用一个因子图（Kschischang等人，2001年）来以图模型的形式表示给定的估计问题。形式上，因子图是一个二分图$G = (\mathscr{F}, \Theta, \mathscr{E})$，具有两种节点类型：因子节点$f_i \in \mathscr{F}$和变量节点$\theta_j \in \Theta$。边$e_{ij} \in \mathscr{E}$始终在因子节点和变量节点之间。因子图$G$定义了函数$f(\Theta)$的因子分解为

$$f(\Theta) = \prod_i f_i(\Theta_i) \tag{1}$$

Θ_i 是因子<code style id='3'>f</code>$_i$相邻的变量 θ_j 的集合，并且独立性关系由边<code style id='6'>e</code>$_{ij}$编码：每个因子<code style id='9'>f</code>$_i$ 都是变量 Θ_i中的函数。我们的目标是在(1)中找到最大化变量分配 Θ^*。

$$\Theta^* = \arg\max_{\Theta} f(\Theta) \tag{2}$$

SLAM问题的通用因子图公式如图2所示，其中地标测量<code style id='1'>m</code>、回环闭合约束<code style id='3'>c</code>和里程计测量<code style id='5'>u</code>是因子的示例。请注意，这种公式允许我们的工作支持任意数量变量的通用概率分布或成本函数，从而可以包含T-SAM（Ni等人，2007年）和协同测绘（Kim等人，2010年）中使用的校准参数或空间分离器。

高斯情况

当假设高斯测量模型时

$$f_i(\Theta_i) \propto \exp\left(-\frac{1}{2}\|h_i(\Theta_i) - z_i\|^2_{\Sigma_i}\right) \tag{3}$$

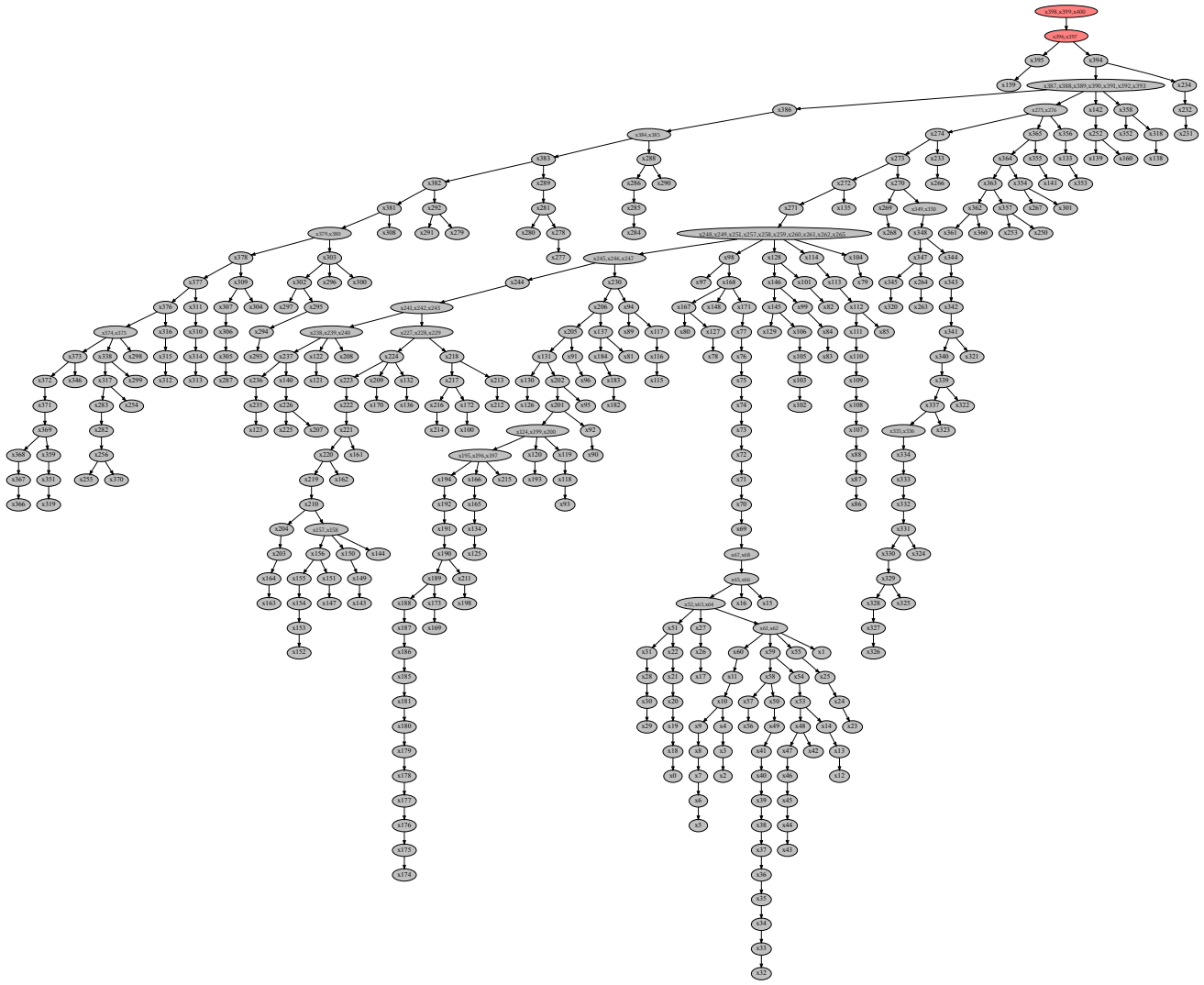


图1： 贝叶斯树数据结构的示例，展示了曼哈顿序列的第400步（有关完整序列与地图的动画，请参见附录A中的扩展1）。我们的增量非线性最小二乘估计算法iSAM2基于将增量因子分解视为编辑对应于解后验概率的图模型，即贝叶斯树。当机器人探索环境时，新的测量通常仅影响树的小部分，因此只需重新计算这些部分。

正如SLAM文献（Smith等人，1987年；Castellanos等人，1999年；Dissanayake等人，2001年）中标准的那样，要最大化的因子化目标函数（2）对应于非线性最小二乘准则

$$\arg\min_{\Theta}(-\log f(\Theta)) = \arg\min_{\Theta}\frac{1}{2}\sum_i \|h_i(\Theta_i)-z_i\|^2_{\Sigma_i} \qquad (4)$$

其中 $h_i(\Theta_i)$ 是测量函数，z_i 是测量值，而 $\|e\|^2_\Sigma \triangleq e^T\Sigma^{-1}e$ 定义为具有协方差矩阵 Σ的平方马氏距离。

在实践中，通常考虑问题 (4) 的线性化版本。对于 (3) 中的非线性测量函数 h_i，高斯-牛顿迭代法或Levenberg-Marquardt算法等非线性优化方法通过求解 (4) 的一系列线性近似来逼近最小值。在非线性求解器的每次迭代中，我们对earize围绕一个线性化点 Θ 以得到一个新的、线性最小二乘问题在 Δ

$$\arg\min_{\Delta}(-\log f(\Delta)) = \arg\min_{\Delta}\|A\Delta-\mathbf{b}\|^2 \qquad (5)$$

其中 $A \in \mathbb{R}^{m\times n}$ 是包含 m行测量值的测量雅可比矩阵，而 Δ 是一个 n-维向量。请注意，协方差 Σ_i 已被吸收到 A 的相应块行中，从而利用

$$\|\Delta\|^2_\Sigma = \Delta^T\Sigma^{-1}\Delta = \Delta^T\Sigma^{-\frac{T}{2}}\Sigma^{-\frac{1}{2}}\Delta = \left\|\Sigma^{-\frac{1}{2}}\Delta\right\|^2 \qquad (6)$$

一旦找到 Δ，新的估计值由 $\Theta\oplus\Delta$ 给出，该值随后被用作非线性优化的下一次迭代中的线性化点。算子 $\oplus$ 通常很简单，是加法，但对于过参数化的表示（例如用于 3D 方向的四元数或计算机视觉中的齐次点表示），则使用基于李群理论（Hall，2000）的指数映射代替。

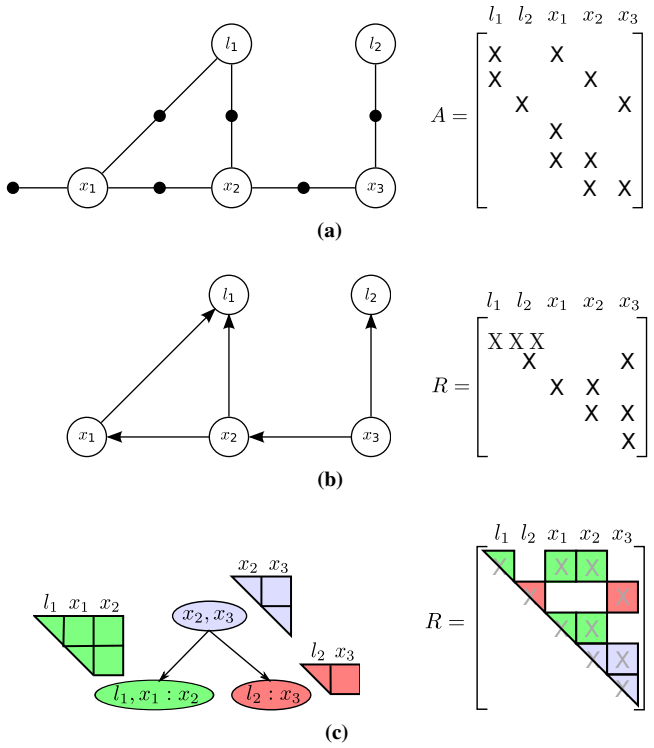


图3： (a) 一个小型SLAM示例的因子图和相关的雅可比矩阵 A，其中机器人位于连续的位姿 $x_1, x_2,$ 和 x_3，对地标 l_1 和 l_2 进行观测。此外，在位姿 x_1 上有一个绝对测量值。(b) 和弦贝叶斯网络和相关的平方根信息矩阵 R，通过使用消除排序 l_1, l_2, x_1, x_2, x_3 从因子图中消除因子得到。最后被消除的变量，这里 x_3，称为根。(c) 贝叶斯树和相关的平方根信息矩阵 R，描述了和弦贝叶斯网络中的团结构。贝叶斯树类似于连接树，但更擅长捕捉稀疏线性代数与图模型推理之间的形式等价性。团及其条件密度与 R 因子的行之间的关联通过颜色表示。

对于这些表示，则使用基于李群理论（Hall，2000）的指数映射代替。

上述矩阵 A 是一个稀疏分块矩阵，其图形模型对应物是一个 高斯 因子图（即原因子图在 Θ 处线性化），与非线性因子图具有完全相同的结构，参见图 3a 中的小型 SLAM 示例。该因子图在 Δ 上定义的概率密度是正态分布

$$P(\Delta) \propto e^{-\log f(\Delta)} = \exp\left\{-\frac{1}{2}\|A\Delta - \mathbf{b}\|^2\right\} \qquad (7)$$

线性系统 $A\Delta - \mathbf{b}$ 的最小值可以直接通过Cholesky分解或QR矩阵分解获得。通过将 Δ 中的导数设为零，我们得到法方程 $A^TA\Delta = A^T\mathbf{b}$。Cholesky分解得到 $A^TA = R^TR$，并在 $R^T\mathbf{y} = A^T\mathbf{b}$ 和 $R\Delta = \mathbf{y}$ 上进行前向和后向替换，首先恢复 $\mathbf{y}$，然后得到实际解，即更新 Δ。或者我们可以跳过法方程，应用QR分解，得到 $R\Delta = \mathbf{d}$，这可以直接通过后向替换求解。请注意 Q 没有显式形成；相反，在分解过程中修改 $\mathbf{b}$ 以获得 $\mathbf{d}$，见Kaess

算法 1 使用直接方程求解器（Cholesky、QR）的 SLAM 平滑解的一般结构。步骤 3-6 可选地可以进行迭代和/或修改以实现 Levenberg-Marquardt 算法。

对每一步中的新测量重复：

1. 添加新测量。
2. 添加并初始化任何新变量。
3. 在当前估计 Θ 处线性化。
4. 使用 QR 或 Cholesky 进行分解。
5. 通过回代求解以获得 Δ。
6. 获得新的估计值 $\Theta' = \Theta \oplus \Delta$。

等(2008)的详细信息。算法1展示了使用直接方法求解SLAM问题平滑公式的必要步骤的总结。

增量和平滑可以通过我们最初的 iSAM 算法（Kaess 等人，2008 年）实现，但在周期性的批量重排序步骤期间才执行重新线性化。如上所述提出的批量解决方案执行不必要的计算，因为它在每一步都解决整个问题，包括所有先前的测量。新测量通常只有局部影响，使地图的远程部分保持未受影响。iSAM 利用这一事实，通过使用新测量增量更新平方根信息矩阵 R。更新是使用 Givens 旋转执行的，并且通常只影响矩阵的一小部分，因此比批量分解便宜得多。然而，随着新变量被追加，变量排序远非最优，并且可能会发生填充。iSAM 执行周期性的批量步骤，在这些步骤中变量被重排序，需要批量分解。该解决方案不是最优的，因为线性化仅在批量步骤期间执行，并且周期性批量步骤的频率是启发式确定的。

3 贝叶斯树

在本节中，我们描述了如何通过直接对图模型进行操作来求解估计问题，而无需将因子图转换为稀疏矩阵然后应用稀疏线性代数方法。

3.1 推理与消元

一个关键洞见是，推理可以被理解为使用消元算法将因子图转换为贝叶斯网络。变量消元（布莱尔和皮尔逊，1993年；考威尔等人，1999年）起源于求解线性方程组，并由高斯在19世纪初首次应用于现代（高斯，1809年）。

在因子图中，消元是通过黑格纳斯和马斯特姆斯（1996）所描述的<样式 id='1'>二分图消元博弈</样式>来完成的。这可以理解为将因子图拆解并转换成一个<样式 id='3'>贝叶斯网络</样式>（珀尔，1988）。通过一次消元一个变量，并将其转换为贝叶斯网络中的一个节点，从而逐步构建该网络。在消元每个变量后，缩减后的因子图定义了剩余变量上的密度。消元变量 的伪代码在算法2中给出。在消元所有变量后，贝叶斯网络的密度由每一步产生的条件概率的乘积定义：

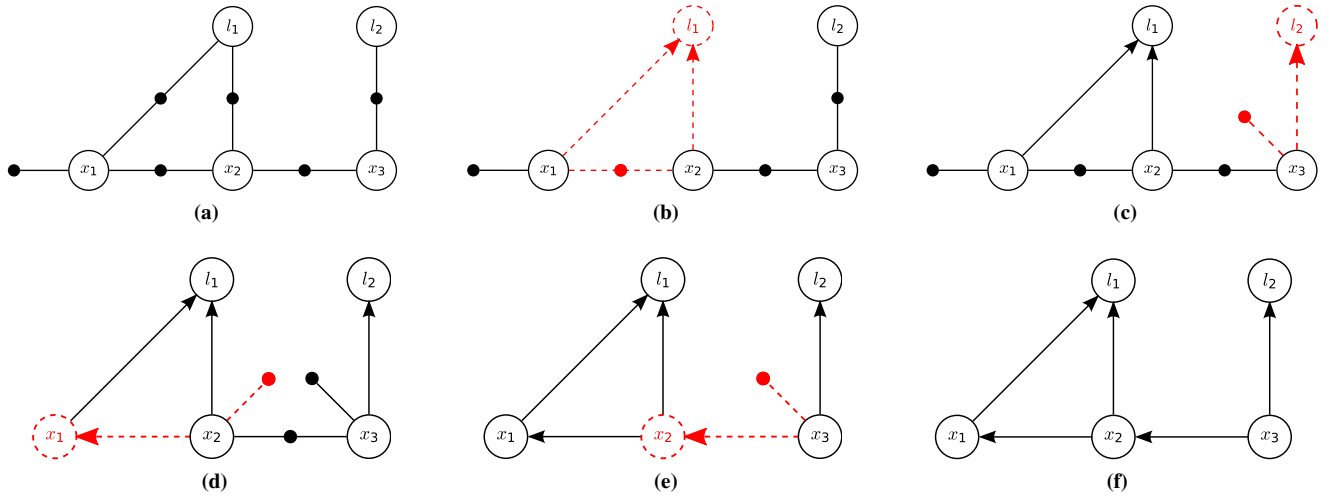


图4： 变量消元过程的步骤，从图3a的因子图开始，到图3b的和弦贝叶斯网络结束。遵循算法2，每一步消除一个变量（虚线红色圆圈），并将所有相邻因子组合成联合分布。通过应用链式法则，将此联合密度转换为条件（虚线红色箭头）和分隔器上的新因子（虚线红色因子）。这个新因子表示一个先验，它总结了被消除变量对分隔器的影响。

算法2 从因子图中消除变量 θ_j。

1. 从因子图中移除所有与 $_i$ 相邻的 $f_i(\Theta_i)$ 因子。
   θ_j。定义 分隔器 S_j 为所有参与这些因子的变量，
   不包括 θ_j。
2. 构造（非归一化的）联合密度 $f_{joint}(\theta_j,S_j) = \prod_i f_i(\Theta_i)$ 作为
   这些因子的乘积。
3. 使用链式法则，对联合密度 $f_{joint}(\theta_j,S_j)$ =进行因子分解
   $P(\theta_j|S_j)f_{new}(S_j)$。将条件概率 $P(\theta_j|S_j)$ 添加到贝叶斯网络中，并将因子 $f_{\theta_j|(S_j)}$ 重新放入因子图中。
   其中 S_{new}是 j 的分离器，即直接通过因子与 相连的变量集合。
   图 3 展示了小规模 SLAM 示例中通过消除操作得到的因子图和
   贝叶斯网络。

在消元每个变量后，缩减后的因子图定义了剩余变量上的密度。消元变量 θ_j 的伪代码在算法2中给出。在消元所有变量后，贝叶斯网络的密度由每一步产生的条件概率的乘积定义：

$$P(\Theta) = \prod_j P(\theta_j|S_j) \tag{8}$$

其中 S_j是 θ_j 的分离器，即直接通过因子与 θ_j 相连的变量集合。图 3 展示了小规模 SLAM 示例中通过消除操作得到的因子图和贝叶斯网络。

For illustration, the intermediate steps of the elimination process are shown in Fig. 4, and we explain the first step here in detail. The factor graph in Fig. 4(a) contains the following six factors: $f(x_1)$, $f(x_1,x_2)$, $f(x_2,x_3)$, $f(l_1,x_1)$, $f(l_1,x_2)$, $f(l_2,x_3)$. The first variable to be eliminated is the first landmark l_1. Following Alg. 2, first we remove all factors involving this landmark ($f(l_1,x_1)$, $f(l_1,x_2)$), and define the separator $S = \{x_1,x_2\}$. Second, we combine the removed factors into a joint factor $f_{joint}(l_1,x_1,x_2)$. Third, we apply the chain rule to split the joint factor into two parts: The first part is a conditional density $P(l_1|x_1,x_2)$ over the eliminated variable given the separator, which shows up as two new arrows in Fig. 4(b). The second part created by the chain rule is a new factor $f(x_1,x_2)$ on the separator as shown in the figure. Note that this factor can also be unary as is the case in the next step when the second landmark l_2 is eliminated and the separator is a single variable, x_3. In all intermediate steps we have both an incomplete factor graph and an incomplete Bayes net. The elimination is complete after the last variable is eliminated and only a Bayes net remains. Speaking in terms of probabilities, the factors $\prod_i f_i(\Theta_i)$ have been converted into an equivalent product of conditionals $\prod_j P(\theta_j|S_j)$.

高斯情况

在高斯因子图中，消元相当于测量雅可比矩阵的稀疏 QR分解。基于链式法则的因子分解 $f_{联合}(\theta_j,S_j) = P(\theta_j|S_j)f_{new}(S_j)$ 在算法2的第3步中可以使用Householder反射或Gram-Schmidt正交化来实现，此时整个消元算法相当于整个测量矩阵A的QR分解。要理解这一点，请注意，对于 $\Delta_j \in \mathbb{R}$ 和 $\mathbf{s}_j \in \mathbb{R}^{n_j}$（组合成长度为 n_j 的向量中的变量集 S_j），因子 $f_{联合}(\Delta_j,\mathbf{s}_j)$ 定义了一个高斯密度

$$f_{joint}(\Delta_j,\mathbf{s}_j) \propto \exp\left\{-\frac{1}{2}\left\|\mathbf{a}\Delta_j + \mathbf{A}_S\mathbf{s}_j - \mathbf{b}\right\|^2\right\} \tag{9}$$

在联合密度（9）中，对于给定的{s}$_j$值，通过评估联合密度（9）来获得所需的条件{P}({j}|s)$_j$，其中{a} $\in \mathbb{R}^{m_j}$、{A}$_S \in \mathbb{R}^{m_j \times n}${j}和{b} $\in \mathbb{R}^{m_j}$，以及{m}$_j$是所有与{j}连接的因子的测量行数。

$$P(\Delta_j|\mathbf{s}_j) \propto \exp\left\{-\frac{1}{2}(\Delta_j + \mathbf{r}\mathbf{s}_j - d)^2\right\} \tag{10}$$

with $\mathbf{r} \triangleq \mathbf{a}^{\dagger}\mathbf{A}_S$ and $d \triangleq \mathbf{a}^{\dagger}\mathbf{b}$, where $\mathbf{a}^{\dagger} \triangleq \left(\mathbf{a}^T\mathbf{a}\right)^{-1}\mathbf{a}^T$ is the *pseudo-inverse* of $\mathbf{a}$. The new factor $f_{new}(\mathbf{s}_j)$ is obtained by substituting $\Delta_j = d - \mathbf{r}\mathbf{s}_j$ back into (9):

$$f_{new}(\mathbf{s}_j) = \exp\left\{-\frac{1}{2}\left\|\mathbf{A}'\mathbf{s}_j - \mathbf{b}'\right\|^2\right\} \tag{11}$$

在 $\mathbf{A}' \triangleq \mathbf{A}_S - \mathbf{a}\mathbf{r}$ 和 $\mathbf{b}' \triangleq \mathbf{b} - \mathbf{a}d$.

上述是Gram-Schmidt的一步，以密度进行解释，稀疏向量$\mathbf{r}$ 和标量 d 可以被识别为指定贝叶斯网络中的一个联合条件密度，或者等价地，稀疏平方根信息矩阵的一行。通过变量消元得到的Chordal贝叶斯网络结果因此等同于通过变量消元获得的平方根信息矩阵，如图3b所示。注意，或者可以从信息矩阵 A^TA 开始执行不完整的Cholesky分解。

通过一次从树叶到树根计算最优分配 Δ^* 来定义所有函数，然后一次向下检索所有面变量的最优分配，这些变量共同组成 Δ 变量。第一次遍历在贝叶斯树构建过程中已经完成，并由每个团块相关的条件密度表示。第二次遍历根据(10)从根开始恢复最优分配，通过求解

$$\Delta_j = d - \mathbf{r}\mathbf{s}_j \tag{12}$$

对于稀疏线性代数中称为回代（backsubstitution）的每一个变量 Δ_j。

3.2 创建贝叶斯树

在本节中，我们介绍一种新的数据结构，贝叶斯树，它源自通过消除操作得到的贝叶斯网络，旨在更好地捕捉与线性代数的等价性，并支持递归估计中的新算法。通过消除/分解操作得到的贝叶斯网络是和弦的，可以将其转换为树状图模型，其中优化和边缘化操作更为容易。贝叶斯树是有向树，其节点表示底层和弦贝叶斯网络中的团 C_k 。贝叶斯树与连接树（考威尔等人，1999）相似，但贝叶斯树是有向的，并且在编码因子化概率密度方面更接近贝叶斯网络。特别地，我们为每个节点定义一个条件密度$P(F_k|S_k)$，其中分离器 S_k 是团C_k 与其父团 Π_k的交集$C_k \cap \Pi_k$ ，而前沿变量 F_k 是剩余变量，即$F_k \triangleq C_k \setminus S_k$。我们写作$C_k = F_k : S_k$。这导致贝叶斯树定义的变量 Θ上的联合密度$P(\Theta)$的表达式为，

$$P(\Theta) = \prod_k P(F_k|S_k) \tag{13}$$

算法3 从和弦贝叶斯网络（chordal Bayes net）中创建贝叶斯树，该网络由消元（elimination）产生（算法2）。

初始化（算法2）。

对于贝叶斯网络中的每个条件密度 $P(\theta_j|S_j)$，在消元顺序：

如果没有父节点（$S_j = \{\}$）

　则开始一个新的根团簇F_r，其中包含 θ_j

else

　识别包含 S_j 的第一个被消除变量的父团簇C_p 作为一个前沿变量

　如果父团簇 C_p 的节点 $F_p \cup$ 和 S_p 等于条件团簇的分离器节点 S_j 则

　　将条件式插入团C_p

　else

　　以C_p 为父节点开始新的团C'，其中包含 θ_j

其中对于根节点F_r ，分离器为空，即它是对根变量的简单先验$P(F_r)$。根据贝叶斯树的定义，团C_k 的分离器S_k 始终是父团的子集 Π_k，因此图中的有向边与贝叶斯网络中的语义意义相同：条件化。

每个和弦贝叶斯网络都可以通过发现其团来转化为树。在弦图中发现团使用Tarjan和Yannakakis（1984）提出的最大基数搜索算法，该算法以反向消除顺序来发现贝叶斯网络中的团。将贝叶斯网络转化为贝叶斯树的算法总结在算法3中。

高斯情况

在高斯情况下，贝叶斯树与平方根信息因子密切相关。图3a中的小型SLAM示例的贝叶斯树显示在图3c中。贝叶斯树的每个团都包含给定分离变量的团变量的条件密度。所有条件密度共同构成图3c右侧显示的平方根信息矩阵，其中团与矩阵行之间的分配通过颜色显示。请注意，一个贝叶斯树可以对应于多个不同的平方根信息因子，因为任何节点的子节点可以任意排序。这种整体变量顺序的变化既不会改变分解的填充也不会改变任何数值，只是它们在矩阵中的位置。

3.3 增量推理

我们证明了增量推理对应于贝叶斯树的简单编辑，这同时也为原本抽象的增量矩阵分解过程提供了更好的解释和理解。特别是，我们现在将平方根信息矩阵 R 以贝叶斯树 $\mathcal{T}$ 的形式进行存储和计算。当添加新的测量值时，例如一个因子 $f'(x_j,x_{j'})$，仅包含 x_j 和 $x_{j'}$ (分别)以及根的团之间的路径会受到影响。这些团下方的子树不受影响，任何不包含 x_j或 $x_{j'}$ 的其他子树也是如此。受影响的

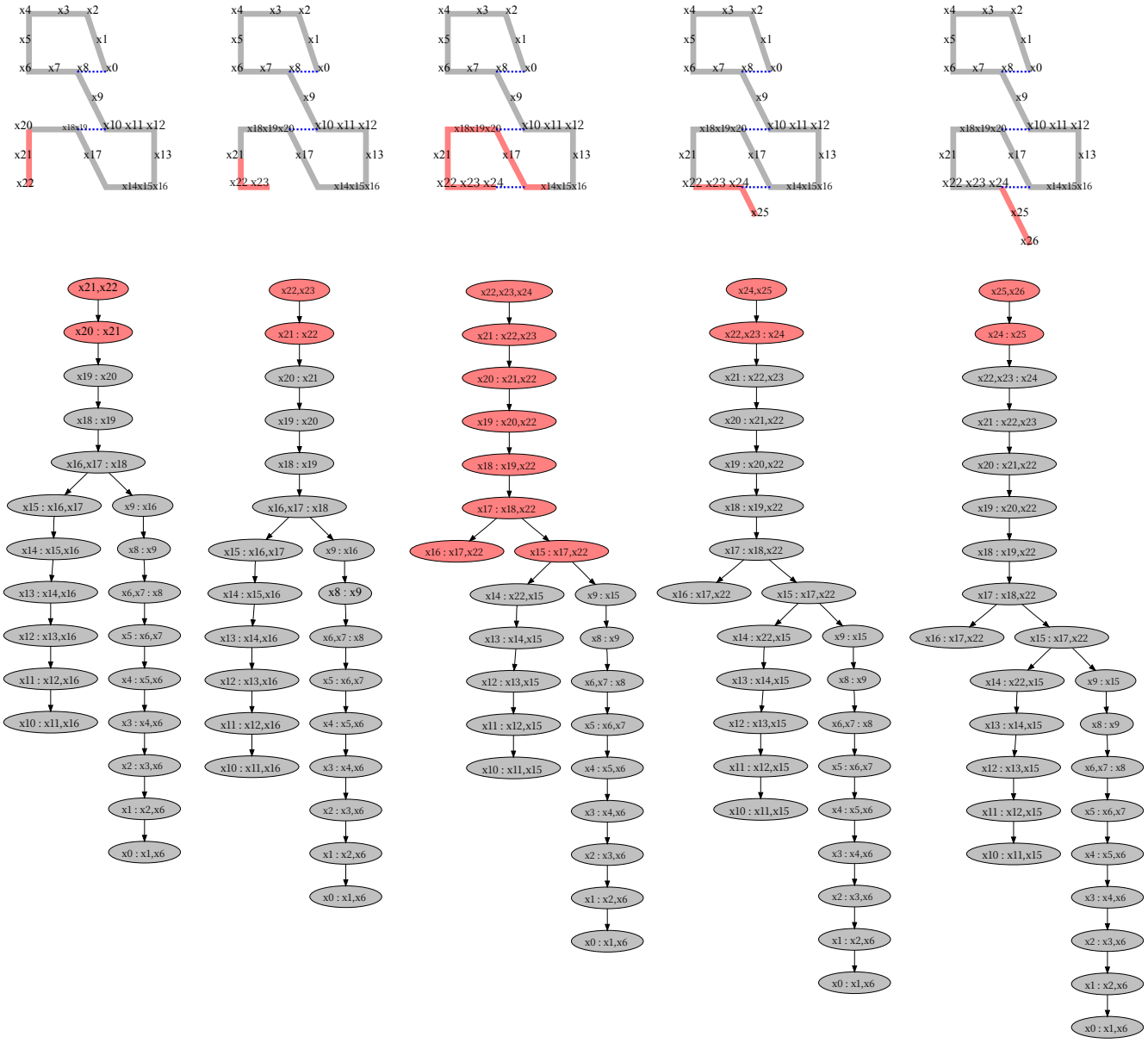


图5： 贝叶斯树的变化：这些列代表一个小型SLAM示例的五个时间步。顶行显示了带有单个机器人姿态的地图，其中回环闭合用虚线蓝色表示。底行描绘了贝叶斯树，其中修改后的团用红色显示。注意中心处的回环闭合会影响变量的一部分，而两个子树保持不变。另见附录A中的扩展1，其中包含完整曼哈顿序列的动画。

贝叶斯树的一部分被转换为因子图，并将新的因子添加到其中。使用新的消元顺序，形成新的贝叶斯树，并将不受影响的子树重新连接。图6展示了如何将这些增量分解/推理步骤应用于图3中的小型SLAM示例，以在 x_1 和 x_3，之间添加新的因子，仅影响树的左分支。使用新因子更新贝叶斯树的全过程在算法4中描述。

为了理解为什么只有树的上部分受到影响，我们来看贝叶斯树的两个重要特性。这些特性直接源于它编码了在消元过程中的信息流。贝叶斯树是形成的

从和弦贝叶斯网络按照反向消元顺序开始。这样，每个团中的变量通过消去这些子团来收集信息从它们的子团。因此，任何团中的信息只向上传播到根节点。其次，一个因子中的信息只有在该因子的第一个变量被消去时才会进入消元过程。结合这两个性质，我们看到一个新的因子不能影响任何不是该因子变量后继的变量。然而，一个作用在具有不同（即独立）路径到根节点的变量上的因子意味着这些路径现在必须重新消去以表达它们之间的新依赖关系。

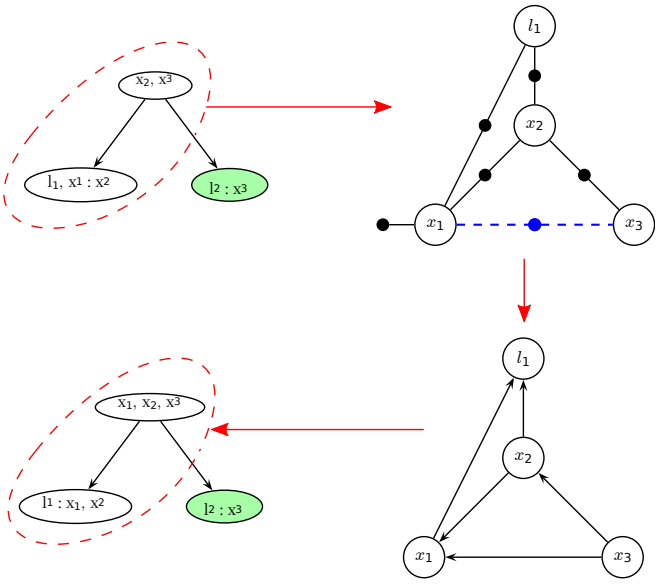


图6： 使用新因子更新贝叶斯树，基于图3c中的示例。贝叶斯树受影响的部分在添加新因子于 x_1 和 x_3 之间时被高亮显示。注意，右分支不受此更改影响。(右上) 从贝叶斯树受影响部分生成并插入新因子（虚线蓝色）的因子图。(右下) 消除因子图后得到的和弦贝叶斯网络。(左下) 从和弦贝叶斯网络创建的贝叶斯树，并将原始贝叶斯树的未修改的右“孤儿”子树重新添加回去。

算法4 使用新因子 $\mathcal{F}'$ 更新贝叶斯树。

输入：贝叶斯树 $\mathcal{T}$，新线性因子 $\mathcal{F}'$
Out: 修改后的贝叶斯树 $\mathcal{T}'$

1. 移除贝叶斯树的顶部，并将其重新解释为因子图：
   (a) 对于每个受新因子影响的变量，移除相应的团以及所有到根的父节点。对应团以及所有到根的父节点。
   (b) 存储被移除团的无父节点子树 $\mathcal{T}_{orph}$。
2. 将新因子 $\mathcal{F}'$ 添加到生成的因子图中。
3. 重新排序因子图的变量。
4. 消除因子图（算法2）并创建一个新的贝叶斯树（算法3）。
5. 将孤儿 $\mathcal{T}_{orph}$ 重新插入新的贝叶斯树。

3.4 增量变量排序

选择一个好的变量顺序对于稀疏矩阵解的效率至关重要，贝叶斯树方法也是如此。变量的最优顺序可以最小化填充，填充是指在消元过程中在平方根信息矩阵中创建的额外条目。在贝叶斯树中，填充转化为更大的团大小，从而导致更慢的计算。通常无法完全避免填充，除非原始贝叶斯网络已经是和弦的。虽然对于一般问题，找到导致最小填充的变量顺序是NP难的（Arnborg等人，1987年），但通常使用启发式算法，如Davis等人（2004年）提出的列近似最小度（CO-LAMD）算法，这些算法可以提供接近

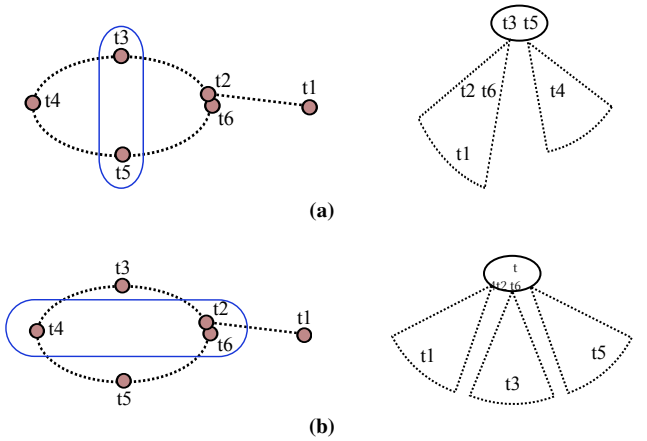


图7： 对于一个带有回环闭合的轨迹，左侧显示了基于嵌套分解的两种不同最优变量排序，要消除的变量用蓝色标记。对于增量更新，策略并不相同，这可以从右侧相应的贝叶斯树中看出。添加与t_6连接的因子将影响 (a) 左子树和根节点，(b) 仅根节点。在后一种情况下，增量更新预计会更快。

以获得许多批量问题的最优排序。

在贝叶斯树中进行增量推理时，变量可以在每次增量更新时重新排序，从而无需定期进行批量重新排序。这一点在(Kaess等人，2008)中并未被理解，因为只有在图模型框架下才显而易见，而矩阵则不然。受影响的贝叶斯树部分，即需要对变量进行重新排序的部分通常很小，因为新的测量通常只影响状态空间的一小部分，而状态空间由估计问题的变量表示。为这部分变量找到最优排序并不一定能提供整体最优排序。然而，我们观察到某些增量排序能提供良好解，与批量应用COLAMD相当。

要理解局部变量排序如何影响后续更新的成本，考虑图7中的简单循环示例。对于简单循环，嵌套分解(Lip- ton和Tarjan，1979)提供最优排序。第一次切割可以：(a)不包含循环闭合，或(b)包含循环闭合，两种解决方案在填充方面是等效的。但在增量情况下存在显著差异：对于(a)中的垂直切割，它不包含最新的变量t_6，，该变量最终会出现在树的下部，导致下一次更新步骤中需要更大范围的树进行变更。另一方面，(b)中的水平切割包含最新的变量，将其推至根节点，因此导致下一次步骤中的变更更小、更高效。

将 COLAMD 局部应用于正在重新计算的树的部分时，会出现类似的问题。在 SLAM 设置中，我们可以预期一组新的测量值将与一些最近观测到的变量连接，无论是仍在传感器范围内的地标，还是通过里程计测量连接的前一个机器人位姿。

将新测量值纳入考虑的预期成本，即更新中受影响的子树的大小，如果这些变量更靠近根节点，将会更低。局部应用 COLAMD 并未考虑这一点，而只是针对当前步骤最小化填充。

为了在后续步骤中实现更快的更新，我们因此提出了一种增量变量排序策略，该策略强制将最近访问的变量置于排序的末尾。我们使用约束 COLAMD (CCOLAMD) 算法（Davis 等人，2004 年）来强制将最近访问的变量置于末尾，同时仍然提供良好的整体排序。后续更新将仅影响树的一小部分，因此可以预期在大多数情况下是高效的，除了大型回环闭合的情况。

我们在图 8 中通过将其与简单地应用 COLAMD 的朴素方法进行比较，评估了我们提出的约束排序策略的价值。图 8 的顶行显示了一个彩色编码的曼哈顿模拟数据集轨迹（Olson 等人，2006 年）。机器人从中心开始，逆时针穿越回环，最后到达左下角。受影响的变量数量从朴素方法（左）显著下降到约束方法（右），因为轨迹中的红色部分（高成本）被绿色（低成本）取代。特别是对于轨迹的左部分，受影响的变量数量比之前小得多，这正如预期的那样，因为良好的排序不会在该区域关闭大型回环。剩余的红色段与轨迹右部分的大型回环闭合相对应。图 8 的第二行显示，与朴素方法相比，约束排序会导致填充量略有增加，而朴素方法本身接近批量排序引起的填充量。底部图形显示，对于朴素方法，受影响的变量数量稳步增加，但对于约束版本，通常保持较低，尽管尖峰表明对于这个问题很可能可以找到更好的增量排序策略。

4 iSAM2算法

在本节中，我们使用贝叶斯树在一个名为iSAM2的新算法中实现机器人应用的在线地图构建。假设存在高斯噪声，该算法会增量地估计一组变量（机器人位置和环境中的地标），同时这两组变量会随时间增长。我们已经展示了如何使用新的线性因子更新贝叶斯树。这留下了如何处理非线性因子以及如何通过仅在需要的地方进行流体重线性化来高效执行此过程的问题，我们称之为流体重线性化。为了进一步提高效率，我们将状态恢复限制在实际发生变化的变量上，从而实现部分状态更新。

4.1 流体重线性化

即时或流体重线性化的核心思想是跟踪每个变量的线性化点的有效性，

算法5 流体重线性化：选定变量的线性化点
根据当前的增量 Δ进行更新。

输入：线性化点 Θ，增量 Δ
Out: 更新线性化点 Θ，标记团M

1. 标记 Δ 中高于阈值 β 的变量：$J=\{\Delta_j \in \Delta | |\Delta_j| \geq \beta\}$。
2. 更新标记变量的线性化点：$\Theta_J := \Theta_J \oplus \Delta_J$。
3. 标记所有涉及标记变量 Θ_J 及其所有祖先的团 M。
   祖先。

算法6 通过重新计算流体重线性化来更新贝叶斯树。
翻译所有受影响的团。请注意，该算法与算法4不同，因为它也包括流体重线性化；结合这两个步骤更高效。

In: 贝叶斯树 $\mathcal{T}$，非线性因子 $\mathcal{F}$，受影响变量 $\mathcal{J}$
Out: 修改后的贝叶斯树 $\mathcal{T}$'

1. 移除贝叶斯树顶部：
   (a) 对于 $\mathcal{J}$ 中的每个受影响变量，移除相应的团及其所有父节点直到根节点。
   (b) 存储已移除团中遗留下来的子树 $\mathcal{T}_{orph}$。
2. 对所有需要重新创建顶部的因子进行relinearize。
3. 添加来自遗留下来的线性因子 $\mathcal{T}_{orph}$。
4. 重新排序变量 , 参见第3.4节。
5. 消除因子图(算法2)并创建一个新的贝叶斯树(算法3)。
6. 将孤儿节点 $\mathcal{T}_{orph}$ 重新插入新的贝叶斯树中。

能够实现，并且仅在需要时进行流体重线性化。这代表了一种与传统线性化/求解方法的不同之处，后者目前代表了直接方程求解器的最先进技术。对于被选择进行流体重线性化的变量，必须从贝叶斯树中移除所有相关信息，并用流体重线性化相应的原始非线性因子来替换。对于被重新消除的团簇，我们还必须考虑从其子树中传递上来的任何边缘因子。在消除过程中缓存这些边缘因子，允许从树的中间重新开始消除过程，而不是必须重新消除整个系统。

我们的流体重线性化算法显示在算法5中。对给定变量进行流体重线性化的决策基于其当前估计与线性化点之间的偏差大于阈值 β。确切地说，变量的不同单位必须被考虑在内，但一个简单的解决方案是取所有阈值的最小值。对于曼哈顿数据集，当阈值为0.1时，几乎可以得到精确解，同时计算成本显著降低，如图9所示。请注意，由于我们为了效率将流体重线性化和更新步骤结合起来，贝叶斯树中的实际变化是在稍后执行的，这与Kaess等人（2010年）提出的原始算法不同。修改后的更新算法显示在算法6中。

4.2 部分状态更新

通过认识到每一步恢复近似精确解并不需要求解所有变量，计算成本可以显著降低。

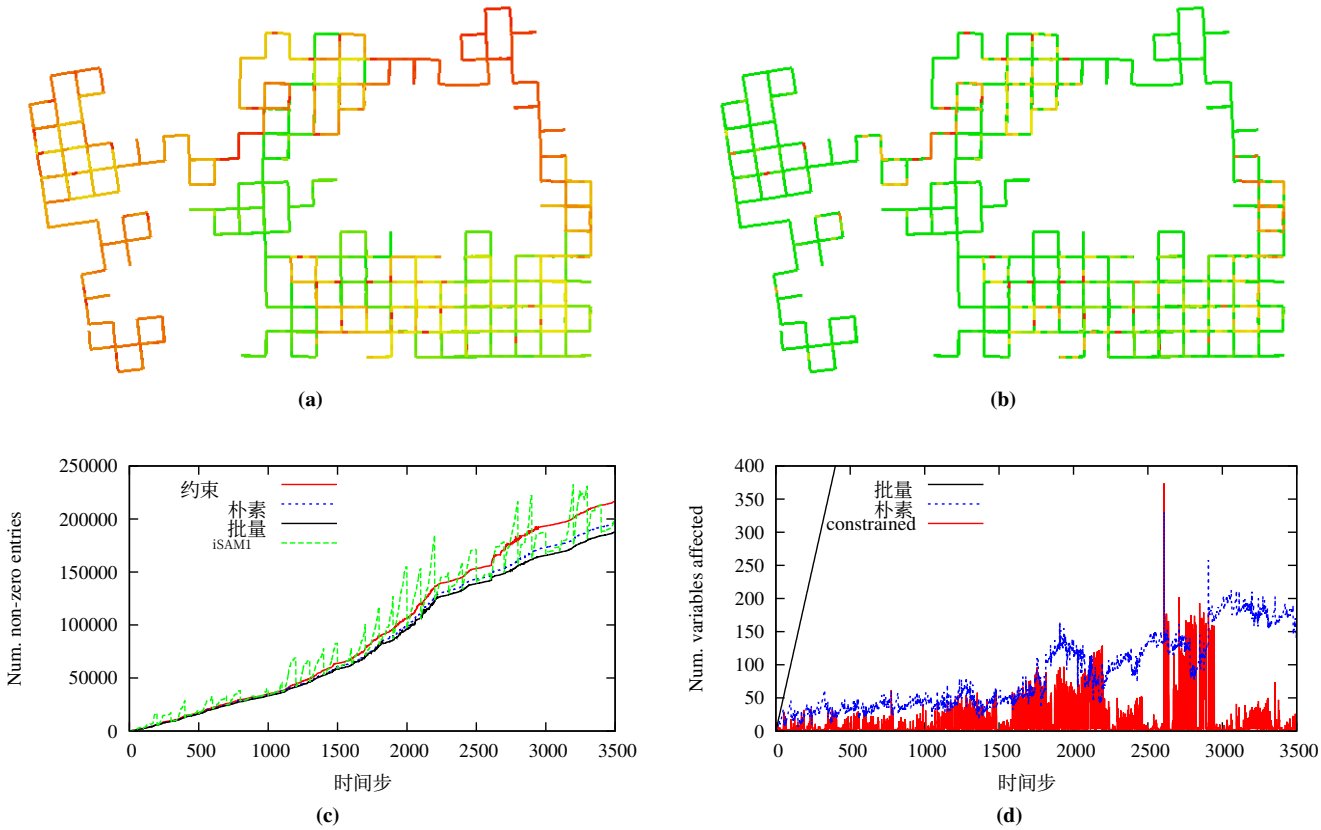


Fig. 8: Comparison of variable ordering strategies using the Manhattan world simulated environment (Olson et al., 2006). By color coding, the top row shows the number of variables that are updated for every step along the trajectory. Green corresponds to a low number of variables, red to a high number. (a) The naive approach of applying COLAMD to the affected variables in each step shows a high overall cost. (b) Forcing the most recently accessed variables to the end of the ordering using constrained COLAMD (Davis et al., 2004) yields a significant improvement in efficiency. (c) Fill-in over time for both strategies as well as the batch ordering and iSAM1. (d) Comparing the number of affected variables in each step clearly shows the improvement in efficiency achieved by the constrained ordering.

贝叶斯树的新因子和重新线性化带来的更新仅影响树的顶部，然而，在此处发生的变量估计变化仍可能进一步向下传播到所有子树。但顶部变化的影响通常有限，因为如果没有闭合大型环路，新的测量值仅具有局部效应，导致空间上遥远的估计部分保持不变。考虑一个包含许多房间的较大建筑物的测绘示例：通常情况下，在某个房间内进行的测量不会影响先前为其他房间获得的估计值。因此，仅求解实际发生变化的变量应显著降低计算成本。

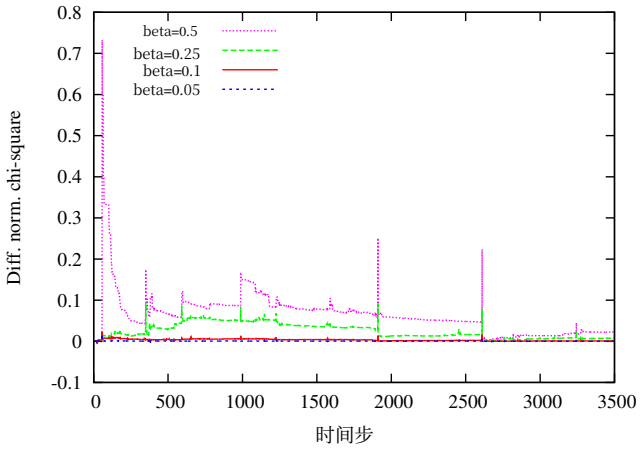
我们如何仅更新实际发生变化的变量，即执行部分状态更新？完整的回代从根节点开始并继续到所有叶节点，获得一个用于更新线性化点 Θ 的增量向量Δ。部分状态更新从求解树修改顶点中包含的所有变量开始。如算法7所示，我们继续处理所有子树，当遇到一个不引用任何变化超过一个较小阈值 α 的变量的团时停止。运行交集属性保证了所有发生变化的变量

算法7 部分状态更新：在非线性情况下求解贝叶斯树
返回一个更新 Δ 到当前线性化点 Θ。

In: 贝叶斯树 $\mathcal{T}$
Out: 更新 Δ
从根团簇$C_r = F_r$开始：

1. 对于当前团簇$C_k = F_k : S_k$
   从局部条件计算 frontal 变量 F_k 的更新 Δ_k
   密度 $P(F_k|S_k)$.
2. 对于所有变量 $\Delta_{k\ j}$ 在 Δ_k 中变化超过阈值 α:
   递归处理包含此类变量的每个后代。

这种情况可能发生在该团簇的任何子树中。请注意，阈值指的是增量向量 Δ 的变化，而不是恢复的增量 Δ 本身的绝对值。Δ 中元素的绝对值可能相当大，因为，如上所述，线性化点 β 只有在达到更大的阈值时才会更新。为简化起见，我们再次对所有变量使用相同的阈值，尽管可以对其进行优化。对于此过程未触及的变量，保持先前的估计 Δ。如图10所示，该方法以显著的计算时间节省获得了近似精确解。



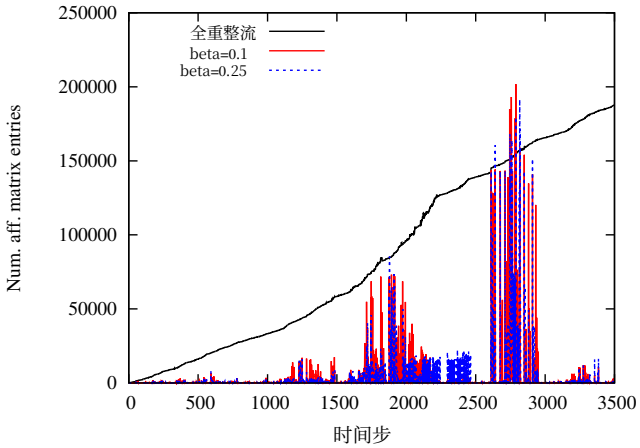
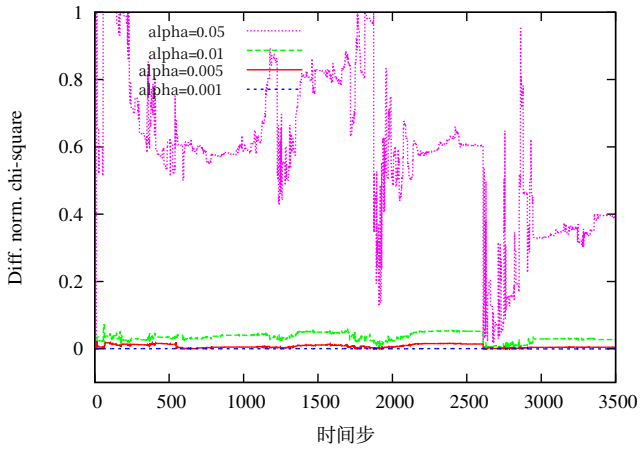


图 9： 重整流阈值 β 如何影响曼哈顿数据集的精度（上）和计算成本（下）。为了提高上图中可读性，减去了最小二乘解的归一化 χ^2 值。阈值为 0。1 对精度没有显著影响，而成本节省是显著的，这从受影响的非零矩阵条目数量可以看出。请注意，对于全重整流，尖峰超出了曲线，因为批量变量排序导致填充有所增加（与图 8 比较）。



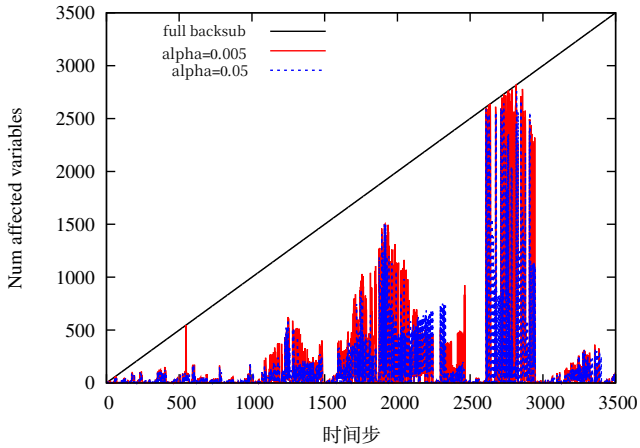


Fig. 10: How the backsubstitution threshold α affects accuracy (top) and computational cost (bottom) for the Manhattan dataset. For readability of the top figure, the normalized χ^2 value of the least-squares solution was subtracted. A small threshold such as 0.005 yields a significant increase in speed, while the accuracy is nearly unaffected.

4.3 Algorithm and Complexity

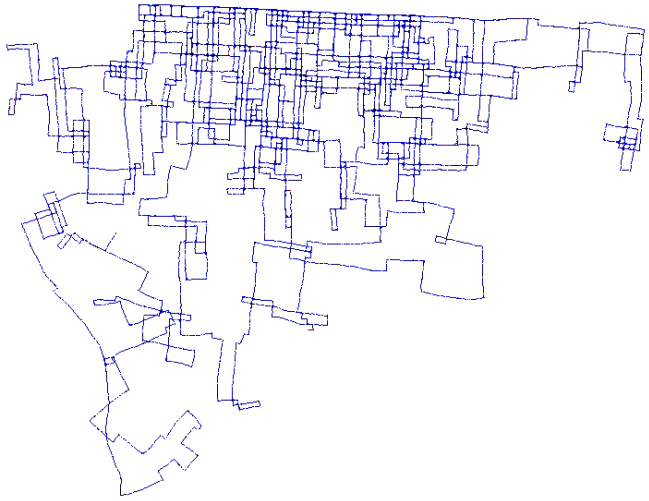
iSAM2 算法总结于算法8。我们算法的目标是，给定一组随时间扩展的非线性约束（由非线性因子 $\mathscr{F}$ 表示），为变量（地图和轨迹）获得估计 Θ。新因子 $\mathscr{F}'$ 可能在任何时间到达，并可能向估计问题添加新变量 Θ'。基于当前的线性化点 Θ，我们在迭代非线性优化方案中作为子程序求解线性化系统。该线性化系统由贝叶斯树 $\mathscr{T}$ 表示。

这里我们提供了一些 iSAM2 的一般复杂度界限。收敛所需的迭代次数通常相当小，特别是由于高斯-牛顿迭代在最小值附近的二次收敛特性。我们这里假设变量的初始化足够接近全局最小值以允许收敛——这是任何直接求解方法的一般要求。对于每项探索任务具有常数数量约束的情况

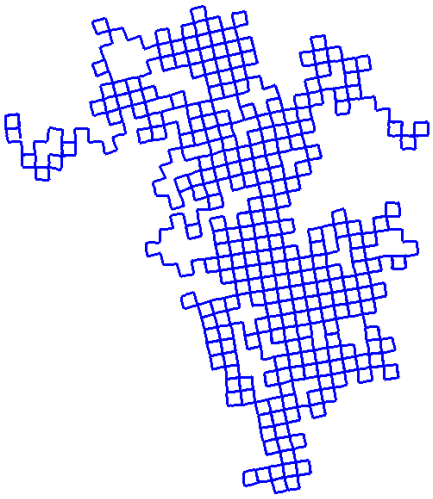
对于每个位姿具有相同数量约束的探索任务，复杂度为 $O(1)$，因为只有树顶部的常数量变量受影响并需要重新消除，并且只有常数量变量需要求解。在回环闭合的情况下，情况变得更加困难，最一般的界限是对于完全因子分解，$O(n^3)$，其中 n 是变量数量（如果存在地标，则包括位姿和地标）。在某些对许多 SLAM 问题成立的假设下，批量矩阵因子分解和回代可以在 $O(n^{1.5})$ (Krauthausen 等人，2006) 内完成。重要的是要注意这个界限不依赖于回环闭合的数量。经验上，复杂度通常远低于这些上界，因为每次步骤中只有矩阵的一小部分需要重新因子分解，如下文所示。

5 与其他方法比较

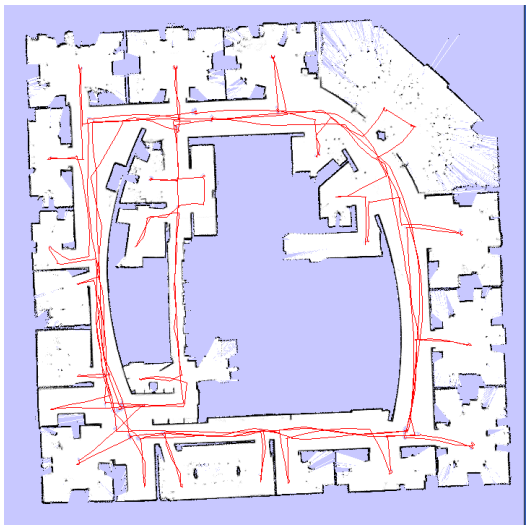
我们将iSAM2与其他最先进的SLAM算法进行了比较，特别是iSAM1算法（Kaess等人，2008年）、HOG-Man（Grisetti等人，2010年）和SPA（Konolige等人，



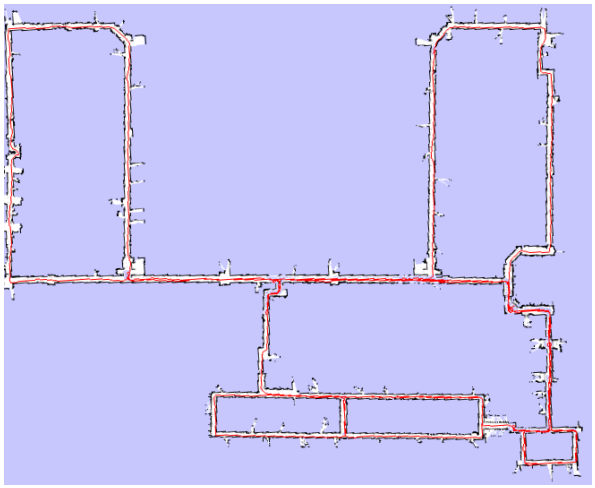
(a) City20000



(b) W10000



(c) Intel



(d) 基利安法院

图11： 二维位姿图数据集，包括模拟数据（City20000、W10000），以及激光雷达数据（基利安法院、Intel）。曼哈顿序列请参见图8。

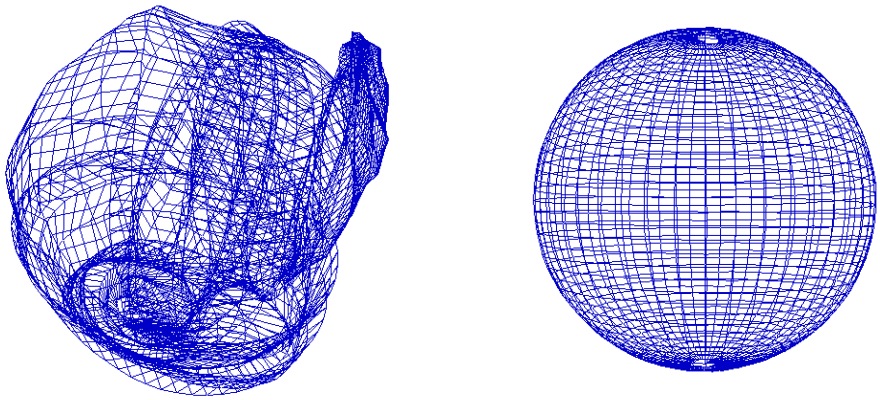
算法 8 一个 iSAM2 算法的步骤，遵循算法 1 中给出的平滑解的一般结构平滑解如算法1所示。

输入/输出：贝叶斯树 $\mathscr{T}$、非线性因子 $\mathscr{F}$、线性化点 Θ、更新 Δ
输入：新的非线性因子 $\mathscr{F}'$, 新的变量 Θ'
初始化：$\mathscr{T} = /0$, $\Theta = /0$, $\mathscr{F} = /0$

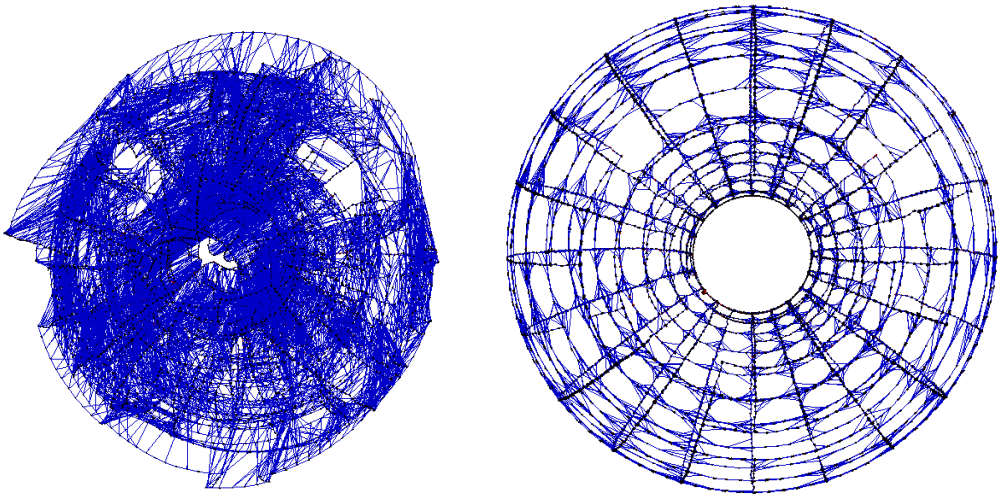
1. 添加任何新的因子 $\mathscr{F} := \mathscr{F} \cup \mathscr{F}'$。
2. 初始化任何新的变量 Θ' 并添加 $\Theta := \Theta \cup \Theta'$。
3. 使用算法5进行流体重线性化，得到标记变量 M，参见第4.1节。节 4.1。
4. 使用算法6重新构建贝叶斯树的顶部 with $\mathscr{J}$ M 和所有受新因子影响的变量。
5. 使用算法7求解增量 Δ，参见第4.2节
6. 由 $\Theta \oplus \Delta$ 给出的当前估计。

2010年）。我们使用各种模拟和现实世界数据集，这些数据集分别如图11、12和13所示，具有不同的尺寸和约束密度，包括仅姿态和带地标的。所有时间结果都是在配备Intel 1。6 GHz i7-720处理器的笔记本电脑上获得的。对于iSAM1，我们使用 http://people.csail.mit.edu/kaess/isam上可用的开源实现版本1.6，并使用标准参数，即在每一步中求解。对于HOG-Man，我们使用http://openslam.org/上可用的svn版本14，并使用命令行选项“-update 1”强制在每一步中求解。对于SPA，我们使用http://www.ros.org/上ROS的svn版本36438，并使用标准参数。

对于iSAM2，我们使用一个单线程的研 cứuC++ 实现，采用Davis等人（2004年）提出的CCOLAMD算法，参数为 $\alpha = 0.001$ 和 $\beta = 0.1$。为了提高效率，每10步执行一次重新线性化。iSAM2的源代码作为gtsam库的一部分提供。



(a) Sphere2500



(b) Torus10000

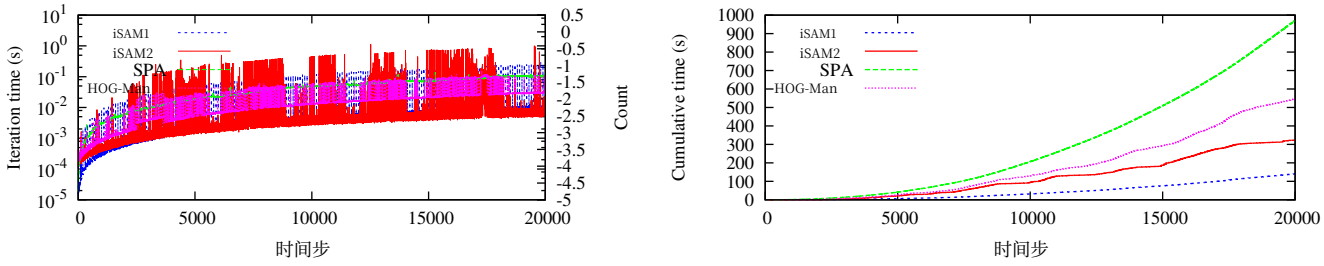
图13： 模拟的3D数据集（sphere2500 和 torus10000，包含在 iSAM1 发行版中）。左侧列显示基于噪声里程计的数据，右侧列显示从 iSAM2 获得的估计。请注意，机器人沿球面和环面表面移动时，会遍历很大的方向范围，因此模拟时分别沿球面和环面表面行驶。

为了提高效率，我们在树的每个节点中使用不完全Cholesky分解代替QR分解。对于3D方向优化，我们使用基于李群理论（Hall，2000年）的指数映射实现的 ⊕算子。我们最初的SAMwork（Dellaert和Kaess，2006年）用于视觉SLAM时使用了欧拉角的局部更新。在这里，作为表示形式，我们使用旋转矩阵（在iSAM2中）和四元数（在iSAM1中）（Grassia，1998年）。我们发现，根据应用场景，每种表示形式都有其自身的优势。

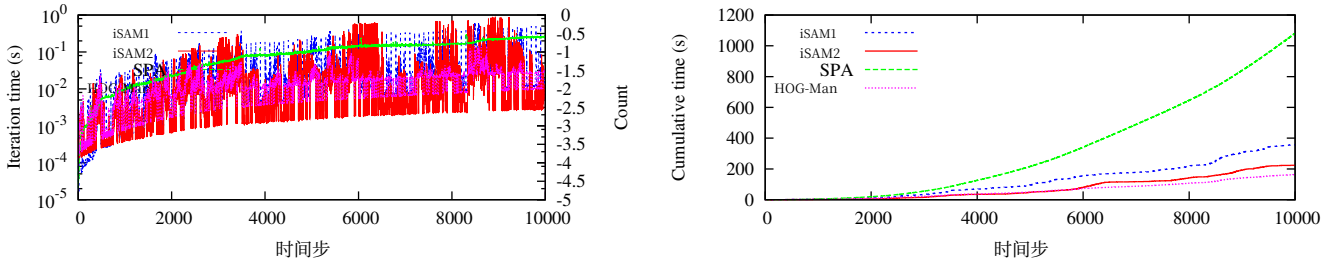
比较不同算法的计算成本并非易事。SLAM 算法的紧致复杂度界限往往不可得。即便复杂度界限可用，它们也未必适合比较，因为涉及的常数在实际应用中可能产生巨大差异。另一方面，算法实现的性能比较则取决于实现本身，以及任何潜在的低效性或不恰当的数据结构选择。因此，我们将不仅讨论从不同实现中获得的计时结果，还将比较底层成本的一些度量，例如稀疏矩阵中有多少条目需要重新计算。但这本身也不是一个完美的度量，因为仅重新计算矩阵的部分内容可能会产生一些无法避免的开销。

5.1 时间

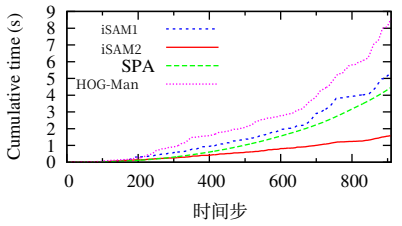
我们在图14中比较了所有数据集上各种算法实现的执行速度，详细结果见表1。结果表明，使用稀疏矩阵分解（SPA、SAM）的批量解决方案很快就会变得昂贵，这突出了增量解决方案的必要性。iSAM1 在稀疏数据集上表现非常好，例如 Manhattan、基利安法院和 City20000，而在具有更密集约束的数据集上性能会下降（约束数量至少是位姿数量的5倍），例如 W10000 和 Intel，这是因为



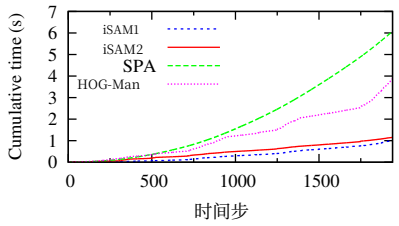
(a) City20000



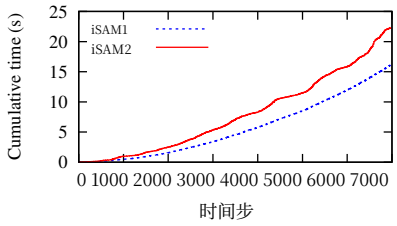
(b) W10000



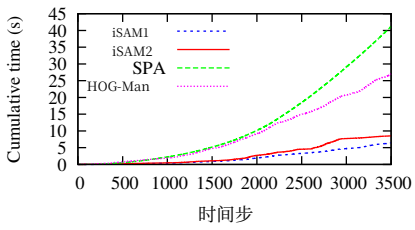
(c) Intel



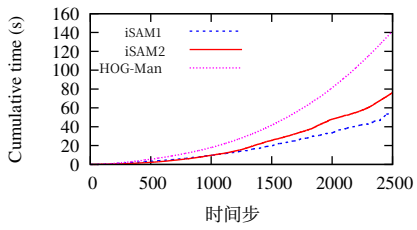
(d) 基利安法院



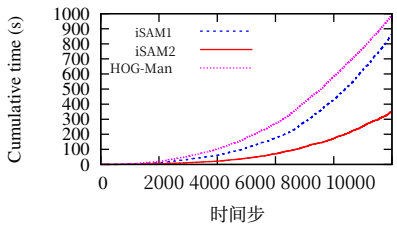
(e)维多利亚公园



(f) 曼哈顿



(g) Sphere2500

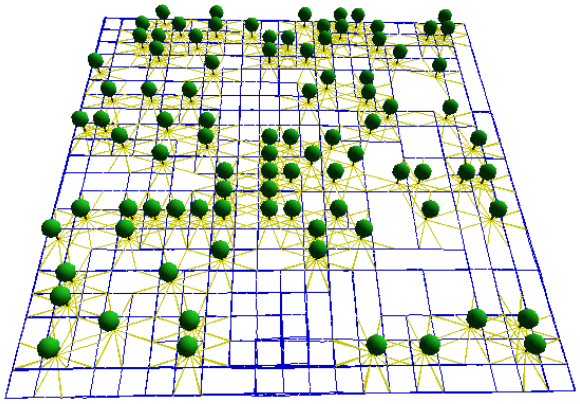


(h) Torus10000

图14： 不同算法在所有数据集上的时间比较，参见图11。左侧列显示每次迭代的时间，右侧列显示累积时间。底部行显示剩余数据集的累积时间。

表1： 不同方法（P：位姿数量，M：测量数量，L：地标数量）的运行时间比较。列出了每步的平均时间、标准差和最大值（毫秒），以及总时间（秒）（最快结果以红色显示）。

	算法			iSAM2		iSAM1		HOG-Man		SPA	
数据集	P	M	L	平均/标准差/最大 [ms]	时间 [s]	平均/标准差/最大 [ms]	时间 [s]	平均/标准差/最大 [ms]	时间 [s]	平均/标准差/最大 [ms]	[s]
City20000	20000	26770	-	16.1 / 65.6 / 1125	323	7.05 / 14.5 / 308	141	27.4 / 27.8 / 146	548	48.7 / 32.6 / 140	977
W10000	10000	64311	-	22.4 / 64.6 / 901	224	35.7 / 58.8 / 683	357	16.4 / 14.9 / 147	164	108 / 75.6 / 287	1081
曼哈顿	3500	5598	-	2.44 / 7.71 / 133	8.54	1.81 / 3.69 / 57.6	6.35	7.71 / 6.91 / 33.8	27.0	11.8 / 8.46 / 28.9	41.1
Intel	910	4453	-	1.74 / 1.76 / 9.13	1.59	5.80 / 8.03 / 48.4	5.28	9.40 / 12.5 / 79.3	8.55	4.89 / 3.77 / 14.9	4.44
基利安法院	1941	2190	-	0.59 / 0.80 / 12.5	1.15	0.51 / 1.13 / 16.6	0.99	2.00 / 2.41 / 11.8	3.88	3.13 / 1.89 / 7.98	6.07
维多利亚公园	6969	10608	151	2.34 / 7.75 / 316	16.3	2.35 / 4.82 / 80.4	16.4	N/A	N/A	N/A	N/A
Trees10000	10000	14442	100	4.24 / 6.52 / 124	42.4	2.98 / 6.70 / 114	29.8	N/A	N/A	N/A	N/A
Sphere2500	2500	4950	-	30.4 / 25.5 / 158	76.0	21.7 / 31.3 / 679	54.3	56.7 / 40.8 / 159	142	N/A	N/A
Torus10000	10000	22281	-	35.2 / 45.7 / 487	352	86.4 / 119 / 1824	864	99.0 / 82.9 / 404	990	N/A	N/A



(a) Trees10000



(b) 维多利亚公园

图12： 带有地标的2D数据集，包括模拟的（Trees10000）和激光雷达数据（维多利亚公园）。

在周期性批量重排序步骤之间进行局部填充（参见图8中心）。请注意，迭代时间图中的峰值是由每100步执行一次的周期性变量重排序引起的，这等效于SPA中执行的批量Cholesky因子化，但对于增量数据结构有一些开销。HOG-Man的性能在大多数数据集上介于SPA和iSAM1和2之间，但在W10000上优于任何其他算法。在更密集的数据集上，性能通常更好，因为层次化操作的优势超过了它们的开销。

iSAM2 一致表现优于 SPA，且与 iSAM1 类似。虽然 iSAM2 通过仅执行部分回代来减少计算量，但流线性化增加了复杂性。流线性化通常影响的变量比线性更新多得多（比较图 8 和图 9），导致贝叶斯树中更大部分需要重新计算。

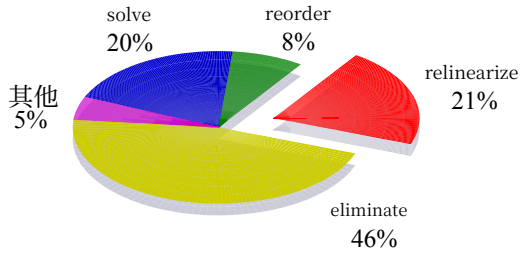


图15： 时间在iSAM2中的花费：W10000数据集上算法各组件花费的时间百分比。

有趣的是，iSAM2 的运行时间峰值与 SPA 跟随，但高出一个数量级，这在图 14ab 中 City20000 和 W10000 的每迭代时间图中很明显。这种差异部分可以解释为 SPA 使用了优化的 CHOLMOD 库（Chen 等人，2008）进行批量 Cholesky 分解，而 iSAM2 底层算法目前还没有这样的库，我们使用的是自己的研究实现。图 15 展示了 iSAM2 中时间如何分配，其中消元是主要部分。

5.2 受影响条目数量

我们还提供一种计算成本度量，它更独立于特定实现，基于受影响的变量数量，以及每次步骤中重新计算的稀疏平方根信息矩阵的条目数量。图10和图9中的底部图表显示了回代过程中受影响的变量数量以及矩阵分解过程中受影响的非零条目数量。红色曲线显示了iSAM2在实现近似精确解的阈值下的成本。与黑色显示的批量解相比，数据清晰地表明，与平方根SAM和SPA相比，iSAM2的计算成本显著降低。

在iSAM2中，相应的平方根信息因子的填充仍然接近批量解，如图8所示。同一图表还显示，对于iSAM1，填充在周期性批量步骤之间显著增加，因为变量每100步才重新排序一次。这种局部填充解释了在约束更密集的数据集（如W10000）上更高的计算成本。由于增量变量排序，iSAM2的填充没有出现显著的局部变化。

5.3 准确率

我们现在关注每个SLAM算法的解的准确性。评估准确性的方法有很多种。我们选择了归一化的 χ^2 度量方法，它量化了最小二乘拟合的质量。归一化的 χ^2 定义为 $\frac{1}{m-n}\sum_i \|h_i(\Theta_i) - z_i\|^2_{\Lambda^i}$，其中分子是(4)中加权平方误差的总和，$m$ 是测量次数，n 是自由度数量。归一化的 χ^2 度量方法衡量约束条件被满足的程度，对于从正态分布中采样的大量测量值，接近1。

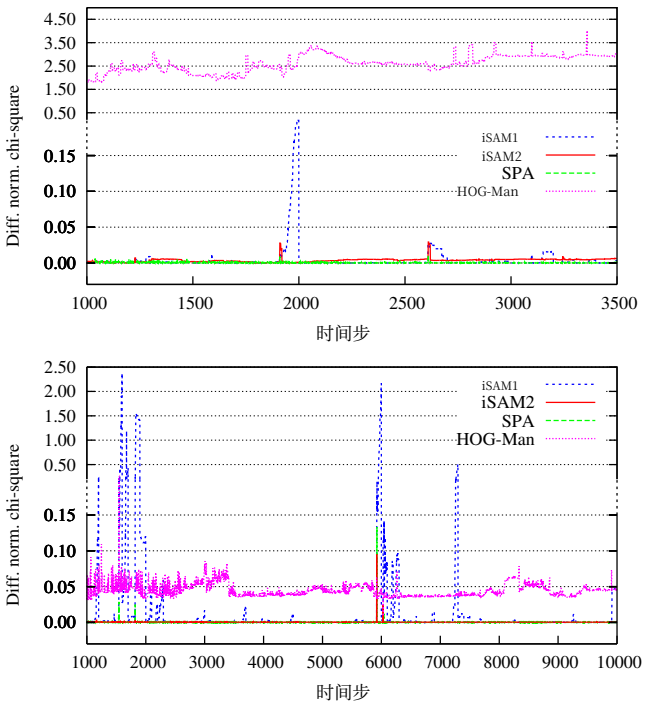


图16：对不同算法在曼哈顿世界（顶部）和W10000数据集（底部）进行逐步质量比较。为提高可读性，显示归一化 χ^2 与最小二乘解的差异（即由$y = 0$给出的真实值）。

ing 1 for a large number of measurements sampled from a normal distribution.

图16中的结果表明，iSAM2解非常接近真实值。真实值是通过在每个步骤中迭代直到收敛获得的均方误差估计。由于仅在每10步进行重新线性化，因此存在小的峰值，这是与计算速度的权衡。iSAM1在误差上显示出更大的峰值，这是由于仅在每100步进行重新线性化。HOG-Man是一种近似算法，表现出始终较大的误差，即使对生成的地图进行视觉检查时也只显示出轻微的扭曲。对于更密集的数据集（如W10000），精度有所提高，但仍然不如iSAM2。

6 相关工作

SLAM 问题的第一个平滑方法由 Lu 和 Milios (1997) 提出，其中估计问题被表述为机器人姿态之间的约束网络。第一个解决方案使用矩阵求逆实现 (Gutmann 和 Nebel, 1997)。此后，基于弛豫 (Duckett 等人，2002；Bosse 等人，2004；Thrun 等人，2005)、梯度下降 (Folkesson 和 Christensen, 2004, 2007)、预条件共轭梯度 (Konolige, 2004；Dellaert 等人，2010)、多级弛豫 (Frese 等人，2005)，以及信念传播（Ranganathan 等人，2007）。

直接方法，如 QR 和 Cholesky 矩阵分解，具有收敛速度更快的优点，至少在提供良好初始化的情况下。它们最初被忽视，因为朴素密集实现过于昂贵。Dellaert (2005) 首次提出了 SLAM 的有效稀疏分解，但现在已广泛使用 (Dellaert 和 Kaess, 2006；Frese, 2006；Folkesson 等人，2007；Kaess 等人，2008；Mahon 等人，2008；Grisetti 等人，2010；Konolige 等人，2010；Strasdat 等人，2010)。平方根 SAM (Del- laert, 2005；Dellaert 和 Kaess, 2006) 通过在每一步使用 Levenberg-Marquardt 算法对完整、自然稀疏的信息矩阵进行 Cholesky 分解来执行平滑。Konolige 等人 (2010) 最近提出了使用 Cholesky 分解的稀疏姿态调整 (SPA)，它引入了可继续的 Levenberg-Marquardt 算法，并专注于信息矩阵的快速设置，这通常是批量分解中最昂贵的部分。

平滑与计算机视觉中的运动结构（Hartley 和 Zisserman，2000）和束调整（Triggs 等人，1999）密切相关。束调整和 SLAM 的平滑公式都保留估计问题中的所有位姿和地标（位姿图 SLAM 是一个特例，省略了地标）。束调整和 SLAM 之间的关键区别在于，束调整通常解决批量问题，而机器人应用需要在线解决方案，因为数据是连续收集的。我们的 iSAM2 算法实现了在线束调整，至少对于某些合理大小的数据集而言。创建一个“永久 SLAM 引擎”以无限期运行的问题仍然悬而未决。请注意，对于基于激光测距的应用，每帧的地标数量通常远低于视觉 SLAM 应用（Eade 和 Drummond，2007；Konolige 和 Agrawal，2008；Paz 等人，2008；Strasdat 等人，2010）。然而，最近研究表明，在许多情况下，平滑已成为视觉 SLAM 的首选方法（Strasdat 等人，2010）。

在增量设置中，通过应用矩阵分解更新，可以避免批量优化的成本，首批SLAM应用出现在(Kaess等人，2007年；Folkesson等人，2007年；Wang，2007年；Mahon等人，2008年)。iSAM算法(Kaess等人，2007年)使用Givens旋转执行增量更新，并定期进行批量分解和非线性化步骤。(Folkesson等人，2007年)仅保留一段简短的机器人姿态历史，避免了重排序问题。Wang(2007年)提到Cholesky更新作为仅包含地标的D-SLAM信息矩阵的选项。iSAM2与这些方法类似，但具有优势，即重排序和非线性化可以在每一步中增量执行。请注意，iSAM2并非简单应用现有方法（如矩阵分解更新），而是引入了一种全新的算法来解决随时间增长的稀疏非线性最小二乘问题。

Olson 等人 (2006) 的相对公式表达相对于先前姿态的姿态，取代了传统

全局公式。所得雅可比矩阵的条目显著更多，但通过随机梯度下降高效求解。相对公式避免了初始化不良问题中的局部最小值。Grisetti 等人 (2007) 提出的分层扩展 TORO 通过显著减少两个任意节点之间的最大路径长度来提供更快的收敛。然而，平移和旋转的分离会导致不精确的解 (Grisetti 等人，2010)，这对 3D 应用尤其成问题。

相对公式也在不同层面上被使用，例如将问题拆分为子地图，比如 Atlas (Bosse 等人，2004) 或 Tectonic SAM (Ni 等人，2007；Ni 和 Dellaert，2010)。在某种程度上，iSAM2 提供了拆分为由不同子树表示的子地图的分离，尽管它们并不是完全分离的，因为新的测量可以随时改变该拓扑结构，因此复杂性并没有明确限制。

Sibley 等人 (2009) 将相对公式与局部限制优化相结合，该优化操作在一个类似于 Howard 等人 (2006) 的流形上。将优化限制在局部区域允许在常数时间内进行更新。生成的地图在局部上是准确的，但只能离线获得全局度量地图。虽然这种地图对某些应用来说是足够的，但我们认为诸如规划之类的任务需要在每个步骤中都提供准确的全球度量地图：例如，要检测/决定是否在 A 和 B 之间可能存在一条未探索的直接路径（如门），需要全局度量信息。在 iSAM2 中，我们因此专注于提高全局度量地图的在线恢复效率——然而，我们的更新步骤不是常数时间，对于大型回环闭合，可能会变得与批量解决方案一样昂贵。

Grisetti 等人 (2010) 最近提出了 HOG-Man，这是一种使用 Cholesky 因子分解的层次化位姿图公式，它在不同细节级别上表示估计问题。计算工作集中在最详细级别上受影响的区域，而任何全局效应都会传播到较粗的级别。特别是对于密集序列，与 iSAM2 相比，HOG-Man 表现良好，但只提供近似解。

Paskin (2003) 提出的薄连接树滤波器 (TJTF) 提供了一种直接基于图模型的增量式解决方案。对于 SLAM 问题的滤波版本，会维护一个增量式连接树，并选择性地省略数据，以保持数据结构稀疏（滤波会导致填充）并使求解的复杂度可控。相比之下，iSAM2 解决完整的 SLAM 问题，并且不省略任何信息。此外，贝叶斯树的构建与连接树不同，连接树首先形成一个团状图，然后找到一个生成树。而贝叶斯树则基于给定的变量排序，类似于矩阵分解。虽然与固定变量排序的平方根信息矩阵相比，贝叶斯树因为团状子树的顺序可以改变而获得了一些灵活性。

Frese (2006) 的 Treemap 在内部执行 QR分解 Treemap is also closely related to the junction tree, though the author ap- proached the subject “fro m a hierarchy-of-regions and linear-equation-solving perspective” Frese (2006). Our work for- malizes this connection in a more comprehensive waythrough the Bayes tree data structure.

7 结论

我们介绍了一种新的数据结构——贝叶斯树，它为现有图模型推理算法和稀疏矩阵因子分解方法提供了算法基础，使我们能够获得新的见解。这些见解促使我们提出了iSAM2，这是一种用于移动机器人中出现的非线性最小二乘问题的完全增量式算法。我们的新算法与原始iSAM算法完全不同，因为变量重排序和重新线性化现在都是增量式进行的。相比之下，iSAM只能增量式地更新线性系统，需要定期进行批量步骤来进行重排序和重新线性化。我们以SLAM为例应用了该算法，尽管该算法同样适用于其他增量式推理问题，例如目标跟踪和传感器融合。我们对iSAM2进行了系统评估，并与另外三种最先进的SLAM算法进行了比较。我们预计我们的基于图的新算法也将有助于更好地理解边缘协方差的恢复，因为我们相信以贝叶斯树表示的简单递归算法在形式上等同于(Kaess and Dellaert, 2009)中描述的动态规划方法。基于图的这种结构也适合利用新处理器中可用的并行化能力。

致谢

M. Kaess、H. Johannsson 和 J. Leonard 部分由 ONR 授予的 N00014-06-1-0043 和 N00014-10-1-0936 资助。F. Dellaert 和 R. Roberts 部分由 NSF 授予的 0713162 号项目 “RI: 大规模图模型推理” 资助。V. Ila 部分由西班牙 MICINN 在其研究人力资源流动国家计划下支持。

我们感谢 E.Olson 提供的曼哈顿数据集，感谢 E.Nebot 和 H. Durrant-Whyte 提供的维多利亚公园数据集，感谢 D. Haehnel 提供的 Intel 数据集，以及感谢 G. Grisetti 提供的 W10000 数据集。

参考文献

Arnborg, S., Corneil, D., 和 Proskurowski, A. (1987). 在 *k*-树中寻找嵌入的复杂性。 *SIAM J. on Algebraic and DiscreteMethods*, 8(2):277–284。Blair, J. 和 Peyton, B. (1993)。团树和 clique 树的介绍。在 George, J., Gilbert, J., 和 Liu, J.-H. 编辑的《图论与稀疏矩阵计算，第 *56* 卷》，*IMA Volumes in Mathematics and its Applications*，第 1–27 页。Springer-Verlag, 纽约。

图论与稀疏矩阵计算，第 56 卷《IMA Volumes in Mathematics and its Applications》，第 *1–27* 页。*Springer-Verlag*, 纽约。

Bosse, M., Newman, P., Leonard, J., 和 Teller, S. (2004). 在大规模循环环境中使用Atlas框架进行同步定位与地图构建. 国际机器人研究杂志, 23(12):1113–1139. Brooks, R. (1985). 移动机器的视觉地图构建. 在 *IEEE*国际机器人与自动化会议 *(ICRA)*, 卷2, 第824 – 829页. Castellanos, J., Montiel, J., Neira, J., 和 Tardós, J. (1999). SPmap: 一种用于同步定位与地图构建的概率框架. *IEEE*机器人与自动化汇刊, 15(5):948–953. Chen, Y., Davis, T., Hager, W., 和 Rajamanickam, S. (2008). 算法887: CHOLMOD, 超节点稀疏Cholesky分解和更新/降级. *ACM*数学软件汇刊, 35(3):22:1– 22:14. Cowell, R., Dawid, A., Lauritzen, S., 和 Spiegelhalter, D. (1999). 概率网络与专家系统. 工程与信息科学的统计. Springer-Verlag. Davis, T., Gilbert, J., Larimore, S., 和 Ng, E. (2004). 一种列近似最小度排序算法. *ACM*数学软件汇刊, 30(3):353–376. Dellaert, F. (2005). 平方根 SAM: 通过平方根信息平滑进行同步定位与映射. 在 机器人科学系统 *(RSS)*. Dellaert, F., Carlson, J., Ila, V., Ni, K., 和 Thorpe, C. (2010). 大规模SLAM的子图预条件共轭梯度法. 在 *IEEE/RSJ*国际智能机器人与系统会议 *(IROS)*. Dellaert, F. 和 Kaess, M. (2006). 平方根SAM: 通过平方根信息平滑进行同步定位与映射. 国际机器人研究杂志 , 25(12):1181–1203. Dellaert, F., Kipp, A., 和 Krauthausen, P. (2005). 一种用于多机器人定位与地图构建的多前沿QR分解方法. 在 会议记录 22*nd* *AAAI*人工智能国家会议, 匹兹堡, PA. Dissanayake, M., Newman, P., Durrant-Whyte, H., Clark, S., 和 Csorba, M. (2001). 解决同步定位与地图构建 (SLAM) 问题的方案. *IEEE*机器人与自动化汇刊, 17(3):229–241. Duckett, T., Marsland, S., 和 Shapiro, J. (2002). 快速、在线学习全局一致地图. 自主机器人, 12(3):287– 300. Eade, E. 和 Drummond, T. (2007). 单目SLAM作为合并观测的图. Folkesson, J. 和 Christensen, H. (2004). 图形SLAM - 一种自校正地图. 在 *IEEE*国际机器人与自动化会议 *(ICRA)*, 卷1, 第383–390页. Folkesson, J. 和 Christensen, H. (2007). 使用图形SLAM闭环控制. *IEEE*机器人汇刊, 23(4):731–741. Folkesson, J., Leonard, J., Leederkerken, J., 和 Williams, R. (2007). 使用声纳进行水下导航的特征跟踪. 在 *IEEE/RSJ*国际智能机器人与系统会议 *(IROS)*, 第3678–3684页.

Frese, U. (2006). Treemap: 一种 $O(\log n)$ 算法，用于室内同步定位与地图构建. 自主机器人, 21(2):103–122.

Frese, U., Hansen, P., 和 Olson, S. (2005). 一种用于同时定位与映射的多级松弛算法。 *IEEE Transactions on Robotics*, 21(2):196–207。

高斯，C. (1809)。*TheoriaMotus Corporum Coelestium in Sec-tionibus Conicis Solem Mabientium* [天体运动理论在圆锥截面中围绕太阳运动]。Perthes and Besser，汉堡，德国。英文翻译可在 http://name.umdl.umich.edu/AGG8895.0001.001. 贵族，W. (1973)。使用Givens变换进行最小二乘计算而不需要平方根。应用数学国际杂志，12:329– 336。Gill，P.，Golub，G.，Murray，W.和Saunders，M. (1974)。修改矩阵分解的方法。数学与计算，28(126):505–535。Golub，G.和Loan，C. V. (1996)。矩阵计算。约翰霍普金斯大学出版社，巴尔的摩，第三版。Grassia，F. S. (1998)。使用指数映射进行旋转的实用参数化。图形工具杂志，3:29–48。Grisetti，G.，Kuemmerle，R.，Stachniss，C.，Frese，U.和Hertzberg，C. (2010)。流形上的分层优化用于在线2D和3D映射。在*IEEE*国际机器人与自动化会议 *(ICRA)*，安克雷奇，阿拉斯加。Grisetti，G.，Stachniss，C.，Grzonka，S.和Burgard，W. (2007)。用于使用梯度下降高效计算最大似然图的树参数化。在机器人：科学与系统 *(RSS)*。Gutmann，J.-S.和Nebel，B. (1997)。使用激光扫描导航移动机器人。在自主移动系统，柏林。Springer Verlag。Hall，B. (2000)。李群、李代数和表示：一种初等介绍。Springer。Hartley，R.和Zisserman，A. (2000)。计算机视觉中的多视图几何。剑桥大学出版社。Heggernes，P.和Matstoms，P. (1996)。为稀疏QR分解找到好的列排序。在第二届*SIAM*稀疏矩阵会议。Howard，A.，Sukhatme，G.和Matari´c，M. (2006)。使用流形表示的多机器人映射。*IEEE*会议录 - 多机器人系统特刊，94(9):1360 – 1369。Julier，S.和Uhlmann，J. (2001)。同时定位和地图构建理论的一个反例。在*IEEE*国际机器人与自动化会议 *(ICRA)*，第4卷，第4238–4243页。Kaess，M.和Dellaert，F. (2009)。从平方根信息矩阵中恢复协方差用于数据关联。机器人与自主系统杂志，57:1198–1210。Kaess，M.，Ila，V.，Roberts，R.和Dellaert，F. (2010)。贝叶斯树：概率机器人映射的算法基础。在机器人算法基础国际研讨会，第157–173页，新加坡。Kaess，M.，Johannsson，H.，Roberts，R.，Ila，V.，Leonard，J.和Dellaert，F. (2011)。iSAM2：增量平滑与映射

使用流体重线性化和增量变量reorder .
在*IEEE* 国际机器人与自动化会议 *(ICRA)*，上海，中国。待发表。Kaess, M., Ranganathan, A., 和 Dellaert, F. (2007)。快速增量平方根信息平滑。在 国际人工智能联合会议 *(IJCAI)*，第 2129–2134 页，海得拉巴，印度。Kaess, M., Ranganathan, A., 和 Dellaert, F. (2008)。iSAM：增量平滑与映射。*IEEE* 机器人 *Transactions*，24(6):1365–1378。Kim, B., Kaess, M., Fletcher, L., Leonard, J., Bachrach, A., Roy, N., 和 Teller, S. (2010)。多相对位姿图用于鲁棒协作映射。在 *IEEE* 国际机器人与自动化会议 *(ICRA)*，第 3185–3192 页，安克雷奇，阿拉斯加。Koller, D. 和 Friedman, N. (2009)。 概率图模型：原理与技术。麻省理工学院出版社，剑桥，MA。Konolige, K. (2004)。大规模地图构建。在 会议记录 21th *AAAI* 国家人工智能会议，圣何塞，CA。Konolige, K. 和 Agrawal, M. (2008)。FrameSLAM：从束调整到实时视觉映射。*IEEE* 机器人 *Transactions*，24(5):1066–1077。Konolige, K., Grisetti, G., Kuemmerle, R., Burgard, W., Benson, L., 和 Vincent, R. (2010)。二维映射的稀疏位姿调整。在 *IEEE/RSJ* 国际智能机器人与系统会议 *(IROS)*。Krauthausen, P., Dellaert, F., 和 Kipp, A. (2006)。利用嵌套分解利用局部性进行平方根平滑与映射。在 机器人：科学与系统 *(RSS)*。Kschischang, F., Frey, B., 和 Loeliger, H.-A. (2001)。因子图和和积算法。*IEEE* 信息论 *Transactions*，47(2)。Lipton, R. 和 Tarjan, R. (1979)。广义嵌套分解。*SIAM* 应用数学杂志，16(2):346–358。Lu, F. 和 Milios, E. (1997)。全局一致的范围扫描对齐用于环境映射。自主机器人，第 333–349 页。Mahon, I., Williams, S., Pizarro, O., 和 Johnson-Roberson, M. (2008)。基于视觉回环的 SLAM 高效实现。*IEEE* 机器人 *Transactions*，24(5):1002–1014。Ni, K. 和 Dellaert, F. (2010)。基于多级子图的嵌套分解 SLAM。在 *IEEE/RSJ* 国际智能机器人与系统会议 *(IROS)*。Ni, K., Steedly, D., 和 Dellaert, F. (2007)。Tectonic SAM：精确；外存；基于子图的 SLAM。在 *IEEE* 国际机器人与自动化会议 *(ICRA)*，罗马；意大利。Olson, E., Leonard, J., 和 Teller, S. (2006)。具有较差初始估计的位姿图快速迭代对齐。在 *IEEE* 国际机器人与自动化会议 *(ICRA)*。Paskin, M. (2003)。用于同时定位和映射的薄连接树滤波器。在国际人工智能联合会议 *(IJCAI)*。Paz, L., Pinies, P., Tardós, J., 和 Neira, J. (2008)。手持立体 6 自由度 SLAM。*IEEE* 机器人 *Transactions*，24(5):946–957。Pearl, J. (1988)。 智能系统中的概率推理：似然推理网络。Morgan Kaufmann。Pothen, A. 和 Sun, C. (1992)。使用团树的分布式多重前沿因子分解。在 第五届 *SIAM* 并行会议记录 科学计算中的*lel*处理，第34–40页。工业与应用数学学会。

Ranganathan, A ., Kaess, M ., 和 Dellaert, F . (2007) . Loopy SAM .
在 国际人工智能联合会议 *(IJCAI)*，第 2191–2196 页，印度海得拉巴。

Sibley, G., Mei, C., Reid, I., 和 Newman, P. (2009). 自适应相对束调整。在 机器人科学和系统 *(RSS)*。Smith, R., Self, M., 和 Cheeseman, P. (1987). 用于不确定空间关系的随机地图。在 国际机器人研究研讨会。Strasdat, H., Montiel, J., 和 Davison, A. (2010). 实时单目 SLAM：为什么需要滤波？在 *IEEE* 国际机器人与自动化会议 *(ICRA)*。Tarjan, R. 和 Yannakakis, M. (1984)。用于测试图的弦性、超图的非循环性和选择性地减少非循环超图的简单线性时间算法。 *SIAM* 计算杂志，13(3):566–579。Thrun, S., Burgard, W., 和 Fox, D. (2005)。 概率机器人学。麻省理工学院出版社，马萨诸塞州剑桥。Triggs, B., McLauchlan, P., Hartley, R., 和 Fitzgibbon, A. (1999)。束调整——现代综合。在 Triggs, W., Zisser- man, A., 和 Szeliski, R. 编辑的 视觉算法：理论与实践，LNCS，第 298–375 页。Springer Verlag。Wang, Z. (2007)。 用于同时定位和地图构建的精确稀疏信息滤波器。博士论文，悉尼科技大学。

A 多媒体扩展索引

本文的多媒体扩展位于：http://www.ijrr.org。

扩展	Type	描述
1	视频	贝叶斯树数据结构和地图随着时间的推移而演变，为曼哈顿序列。